

```
1 template<typename TP>
2 struct MCMF{
```

```

3 static const int MAXN=440;
4 static const TP INF=999999999;
5 struct edge{
6     int v,pre;
7     TP r,cost;
8     edge(int v,int pre,TP r,TP cost):v(v),
9         pre(pre),r(r),cost(cost){}
10 };
11 int n,S,T;
12 TP dis[MAXN],PIS,ans;
13 bool vis[MAXN];
14 vector<edge> e;
15 int g[MAXN];
16 void init(int _n){
17     memset(g,-1,sizeof(int)*((n=_n)+1));
18     e.clear();
19 }
20 void add_edge(int u,int v,TP r,TP cost,
21     bool directed=false){
22     e.push_back(edge(v,g[u],r,cost));
23     g[u]=e.size()-1;
24     e.push_back(
25         edge(u,g[v],directed?0:r,-cost));
26     g[v]=e.size()-1;
27 }
28 TP augment(int u,TP CF){
29     if(u==T||!CF) return ans+=PIS*CF,CF;
30     vis[u]=1;
31     TP r=CF,d;
32     for(int i=g[u];~i;i=e[i].pre){
33         if(e[i].r&&e[i].cost&&!vis[e[i].v]){
34             d=augment(e[i].v,min(r,e[i].r));
35             e[i].r-=d;
36             e[i^1].r+=d;
37             if(!(r==d)) break;
38         }
39     }
40     return CF-r;
41 }
42 bool modlabel(){
43     for(int u=0;u<n;++u) dis[u]=INF;
44     static deque<int> q;
45     dis[T]=0,q.push_back(T);
46     while(q.size()){
47         int u=q.front();q.pop_front();
48         TP dt;
49         for(int i=g[u];~i;i=e[i].pre){
50             if(e[i].r&&(dt=dis[u]-e[i].cost)<
51                 dis[e[i].v]){
52                 if((dis[e[i].v]=dt)<dis[q.size()])
53                     q.push_front(e[i].v);
54                 else q.push_back(e[i].v);
55             }
56         }
57     }
58     for(int u=0;u<n;++u)
59         for(int i=g[u];~i;i=e[i].pre)
60             e[i].cost+=dis[e[i].v]-dis[u];
61     return PIS+=dis[S],dis[S]<INF;
62 }
63 TP mincost(int s,int t){
64     S=s,T=t;
65     PIS=ans=0;
66     while(modlabel()){
67         do memset(vis,0,sizeof(bool)*(n+1));

```

```

65 while(augment(S,INF));
66 }return ans;
67 }
68 };

```

3 Graph

3.1 Augmenting Path

```

1 #define MAXN1 505
2 #define MAXN2 505
3 int n1,n2;//n1個點連向n2個點
4 int match[MAXN2]; //屬於n2的點匹配了哪個點
5 vector<int> g[MAXN1]; //圖 0-base
6 bool vis[MAXN2]; //是否走訪過
7 bool dfs(int u){
8     for(int v:g[u]){
9         if(vis[v]) continue;
10        vis[v]=1;
11        if(match[v]==-1||dfs(match[v]))
12            return match[v]=u, 1;
13    }
14    return 0;
15 }
16 int max_match(){
17     int ans=0;
18     memset(match,-1,sizeof(int)*n2);
19     for(int i=0;i<n1;++i){
20         memset(vis,0,sizeof(bool)*n2);
21         if(dfs(i)) ++ans;
22     }
23     return ans;
24 }

```

3.2 Augmenting Path multiple

```

1 #define MAXN1 1005
2 #define MAXN2 505
3 int n1,n2;
4 //n1個點連向n2個點，其中n2個點可以匹配很多邊
5 vector<int> g[MAXN1]; //圖 0-base
6 size_t c[MAXN2];
7 //每個屬於n2點最多可以接受幾條匹配邊
8 vector<int> matchs[MAXN2];
9 //每個屬於n2的點匹配了那些點
10 bool vis[MAXN2];
11 bool dfs(int u){
12     for(int v:g[u]){
13         if(vis[v]) continue;
14         vis[v]=1;
15         if(matchs[v].size()<c[v]){
16             return matchs[v].push_back(u), 1;
17         } else for(size_t j=0;j<matchs[v].size();
18             ++j){
19             if(dfs(matchs[v][j]))
20                 return matchs[v][j]=u, 1;
21         }
22     }
23     return 0;
24 }

```

```

21 }
22 return 0;
23 }
24 int max_match(){
25     for(int i=0;i<n2;++i) matchs[i].clear();
26     int cnt=0;
27     for(int u=0;u<n1;++u){
28         memset(vis,0,sizeof(bool)*n2);
29         if(dfs(u)) ++cnt;
30     }
31     return cnt;
32 }

```

3.3 blossom matching

```

1 #define MAXN 505
2 int n; //1-base
3 vector<int> g[MAXN];
4 int MH[MAXN]; //output MH
5 int pa[MAXN],st[MAXN],S[MAXN],v[MAXN],t;
6 int lca(int x,int y){
7     for(++t;swap(x,y)){
8         if(!x) continue;
9         if(v[x]==t) return x;
10        v[x]=t;
11        x=st[pa[MH[x]]];
12    }
13 }
14 #define qpush(x) q.push(x),S[x]=0
15 void flower(int x,int y,int l,queue<int>&q){
16     while(st[x]!=1){
17         pa[x]=y;
18         if(S[y]==1) qpush(y);
19         st[x]=st[y]=1, x=pa[y];
20     }
21 }
22 bool bfs(int x){
23     iota(st+1, st+n+1, 1);
24     memset(S+1,-1,sizeof(int)*n);
25     queue<int> q; qpush(x);
26     while(q.size()){
27         x=q.front(),q.pop();
28         for(int y:g[x]){
29             if(S[y]==-1){
30                 pa[y]=x,S[y]=1;
31                 if(!MH[y]){
32                     for(int lst;y;lst=x,pa[y])
33                         lst=MH[x],MH[x]=y,MH[y]=x;
34                     return 1;
35                 }
36                 qpush(MH[y]);
37             } else if(!S[y]&&st[y]!=st[x]){
38                 int l=lca(y,x);
39                 flower(y,x,l,q),flower(x,y,l,q);
40             }
41         }
42     }
43     return 0;
44 }
45 int blossom(){
46     memset(MH+1,0,sizeof(int)*n);
47     int ans=0;
48     for(int i=1;i<n;++i)

```

3.4 BronKerbosch

```

1 struct maximalCliques{
2     using Set = vector<int>;
3     size_t n; //1-base
4     vector<Set> G;
5     static Set setUnion(const Set &A, const
6         Set &B){
7         Set C(A.size() + B.size());
8         auto it = set_union(A.begin(),A.end(),B.
9             begin(),B.end(),C.begin());
10        C.erase(it, C.end());
11        return C;
12 }
13 static Set setIntersection(const Set &A,
14     const Set &B){
15     Set C(min(A.size(), B.size()));
16     auto it = set_intersection(A.begin(),A.
17         end(),B.begin(),B.end(),C.begin());
18     C.erase(it, C.end());
19     return C;
20 }
21 static Set setDifference(const Set &A,
22     const Set &B){
23     Set C(min(A.size(), B.size()));
24     auto it = set_difference(A.begin(),A.end
25         (),B.begin(),B.end(),C.begin());
26     C.erase(it, C.end());
27     return C;
28 }
29 void BronKerbosch1(Set R, Set P, Set X){
30     if(P.empty()&&X.empty()){
31         // R form an maximal clique
32         return;
33     }
34     for(auto v: P){
35         BronKerbosch1(setUnion(R,{v}),
36             setIntersection(P,G[v]),
37             setIntersection(X,G[v]));
38         P = setDifference(P,{v});
39         X = setUnion(X,{v});
40     }
41 }
42 void init(int _n){
43     G.clear();
44     G.resize((n = _n) + 1);
45 }
46 void addEdge(int u, int v){
47     G[u].emplace_back(v);
48     G[v].emplace_back(u);
49 }
50 void solve(int n){
51     Set P;
52     for(int i=1;i<n;++i){
53         sort(G[i].begin(), G[i].end());
54         G[i].erase(unique(G[i].begin(), G[i].end()),
55             G[i].end());
56         P.emplace_back(i);
57     }
58 }

```

```

49   BronKerbosch1({}, P, {});
50 }
51 };

```

3.5 graphISO

```

1  const int MAXN=1005,K=30;//K要夠大
2  const long long A=3,B=11,C=2,D=19,P=0
3      xdefaced;
4  long long f[K+1][MAXN];
5  vector<int> g[MAXN],rg[MAXN];
6  int n;
7  void init(){
8      for(int i=0;i<n;++i){
9          f[0][i]=1;
10         g[i].clear(), rg[i].clear();
11     }
12     void add_edge(int u,int v){
13         g[u].push_back(v), rg[v].push_back(u);
14     }
15     long long point_hash(int u){//O(N)
16         for(int t=1;t<=K;++t){
17             for(int i=0;i<n;++i){
18                 f[t][i]=f[t-1][i]*A*P;
19                 for(int j:g[i])f[t][i]=(f[t][i]+f[t-1][j]*B*P)*P;
20                 for(int j:rg[i])f[t][i]=(f[t][i]+f[t-1][j]*C*P)*P;
21             }
22             if(i==u)f[t][i]+=D;//如果圖太大的話，
23             //把這行刪掉，執行一次後f[K]就會是所有
24             //有點的答案
25             f[t][i]%=P;
26         }
27     }
28     return f[K][u];
29 }
30 vector<long long> graph_hash(){
31     vector<long long> ans;
32     for(int i=0;i<n;++i)ans.push_back(
33         point_hash(i));//O(N^2)
34     sort(ans.begin(),ans.end());
35     return ans;
36 }

```

3.6 is planar

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  struct FringeOpposedSubset {
4      deque<int> left, right;
5      FringeOpposedSubset() = default;
6      FringeOpposedSubset(int h) : left{h},
7          right() {}
8  };
9  template<typename T>
10 void extend(T& a, T& b, bool rev = false) {
11     rev ? a.insert(a.begin(), b.rbegin(), b.
12         rend())

```

```

11     : a.insert(a.end(), b.begin(), b.end()
12         );
13 }
14 struct Fringe {
15     deque<FringeOpposedSubset> FOPs;
16     Fringe(int h) : FOPs{{h}} {}
17     bool operator<(const Fringe& o) const {
18         return std::tie(FOPs.back().left.back(),
19             FOPs.front().left.front()) <
20             std::tie(o.FOPs.back().left.back(),
21                 o.FOPs.front().left.front());
22     }
23     void merge(Fringe& o) {
24         o.merge_t_alike_edges();
25         merge_t_opposite_edges_into(o);
26         if (FOPs.front().right.empty())
27             o.align_duplicates(FOPs.back().left.
28                 front());
29         else
30             make_onion_structure(o);
31         if (o.FOPs.front().left.size()) FOPs.
32             push_front(o.FOPs.front());
33     }
34     void merge_t_alike_edges() {
35         FringeOpposedSubset ans;
36         for (auto& FOP : FOPs) {
37             if (!FOP.right.empty()) throw
38                 runtime_error("Exception");
39             extend(ans.left, FOP.left);
40         }
41         FOPs = {ans};
42     }
43     void merge_t_opposite_edges_into(Fringe& o) {
44         while (FOPs.front().right.empty() &&
45             FOPs.front().left.front() > o.
46                 FOPs.front().left.back()) {
47             extend(o.FOPs.front().right, FOPs.
48                 front().left);
49             FOPs.pop_front();
50         }
51     }
52     void align_duplicates(int dfs_h) {
53         if (FOPs.front().left.back() == dfs_h) {
54             FOPs.front().left.pop_back();
55             swap_side();
56         }
57     }
58     void swap_side() {
59         if (FOPs.front().left.empty() ||
60             (!FOPs.front().right.empty() &&
61                 FOPs.front().left.back() > FOPs.
62                     front().right.back())) {
63             swap(FOPs.front().left, FOPs.front().
64                 right);
65         }
66     }
67     void make_onion_structure(Fringe& o) {
68         auto low = &FOPs.front().left, high = &
69             FOPs.front().right;
70         if (FOPs.front().left.front() >= FOPs.
71             front().right.front())
72             swap(low, high);
73         if (o.FOPs.front().left.back() < low->
74             front())
75             throw runtime_error("Exception");
76     }
77     if (o.FOPs.front().left.back() < high->
78         front()) {
79         extend(*low, o.FOPs.front().left, true
80             );
81         extend(*high, o.FOPs.front().right,
82             true);
83         o.FOPs.front().left.clear();
84         o.FOPs.front().right.clear();
85     }
86 }
87 auto lr_condition(int deep) const {
88     bool L = !FOPs.front().left.empty() &&
89         FOPs.front().left.front() >= deep;
90     bool R = !FOPs.front().right.empty() &&
91         FOPs.front().right.front() >= deep;
92     return make_pair(L, R);
93 }
94 void prune(int deep) {
95     auto [left, right] = lr_condition(deep);
96     while (!FOPs.empty() && (left || right))
97     {
98         if (left) FOPs.front().left.pop_front
99             ();
100         if (right) FOPs.front().right.
101             pop_front();
102         if (FOPs.front().left.empty() && FOPs.
103             front().right.empty())
104             FOPs.pop_front();
105         else swap_side();
106         if (!FOPs.empty()) tie(left, right) =
107             lr_condition(deep);
108     }
109 }
110 unique_ptr<Fringe> get_merged_fringe(deque<
111     unique_ptr<Fringe>&& upper) {
112     if (upper.empty()) return nullptr;
113     sort(upper.begin(), upper.end(), [](auto&
114         a, auto& b) { return *a < *b; });
115     for (auto it = next(upper.begin()); it !=
116         upper.end(); ++it)
117         upper.front()->merge(*it);
118     return move(upper.front());
119 }
120 void merge_fringes(vector<deque<unique_ptr<
121     Fringe>>& fringes, int deep) {
122     auto mf = get_merged_fringe(fringes.back()
123         );
124     fringes.pop_back();
125     if (mf) {
126         mf->prune(deep);
127         if (mf->FOPs.size()) fringes.back().
128             push_back(move(mf));
129     }
130 }
131 struct Edge {
132     int from, to;
133     Edge(int from, int to) : from(from), to(to)
134     {}
135     bool operator==(const Edge& o) const {
136         return from == o.from && to == o.to;
137     }
138 };
139 struct Graph {
140     int n = 0;
141     vector<vector<int>> neighbor;

```

```

112     vector<Edge> edges;
113     void add_edge(int from, int to) {
114         if (from == to) return;
115         edges.emplace_back(from, to);
116         edges.emplace_back(to, from);
117     }
118     void build() {
119         sort(edges.begin(), edges.end(), [](
120             const auto& a, const auto& b) {
121             return a.from < b.from || (a.from == b
122                 .from && a.to < b.to);
123         });
124         edges.erase(unique(edges.begin(), edges.
125             end()), edges.end());
126         n = 0;
127         for (auto& e : edges) n = max(n, max(e.
128             from, e.to) + 1);
129         neighbor.resize(n);
130         for (auto& e : edges) neighbor[e.from].
131             push_back(e.to);
132     }
133 };
134 Graph g;
135 vector<int> Deeps;
136 vector<deque<unique_ptr<Fringe>>> fringes;
137 bool dfs(int x, int parent = -1) {
138     for (int y : g.neighbor[x]) {
139         if (y == parent) continue;
140         if (Deeps[y] < 0) { // tree edge
141             fringes.push_back({});
142             Deeps[y] = Deeps[x] + 1;
143             if (!dfs(y, x)) return false;
144         } else if (Deeps[x] > Deeps[y]) { //
145             back edge
146             fringes.back().push_back(make_unique<
147                 Fringe>(Deeps[y]));
148         }
149     }
150     try {
151         if (fringes.size() > 1) merge_fringes(
152             fringes, Deeps[parent]);
153     } catch (const exception& e) {
154         return false;
155     }
156     return true;
157 }
158 bool is_planar() {
159     Deeps.assign(g.n, -1);
160     for (int i = 0; i < g.n; ++i) {
161         if (Deeps[i] >= 0) continue;
162         fringes.clear();
163         Deeps[i] = 0;
164         if (!dfs(i)) return false;
165     }
166     return true;
167 }
168 int main() {
169     int n, m, u, v;
170     cin >> n >> m;
171     for (int i = 0; i < m; ++i) {
172         cin >> u >> v;
173         g.add_edge(u, v);
174     }
175     g.build();
176     cout << (is_planar() ? "YES" : "NO") <<
177         endl;

```

```

169 return 0;
170 }

```

3.7 KM

```

1 #define MAXN 405
2 #define INF 0x3f3f3f3f3f3f3f3f
3 int n; // 1-base · 0表示沒有匹配
4 LL g[MAXN][MAXN]; //input graph
5 int My[MAXN], Mx[MAXN]; //output match
6 LL lx[MAXN], ly[MAXN], pa[MAXN], Sy[MAXN];
7 bool vx[MAXN], vy[MAXN];
8 void augment(int y){
9     for(int x, z; y; y = z){
10         x=pa[y], z=Mx[x];
11         My[y]=x, Mx[x]=y;
12     }
13 }
14 void bfs(int st){
15     for(int i=1; i<=n; ++i)
16         Sy[i] = INF, vx[i]=vy[i]=0;
17     queue<int> q; q.push(st);
18     for(;;){
19         while(q.size()){
20             int x=q.front(); q.pop();
21             vx[x]=1;
22             for(int y=1; y<=n; ++y) if(!vy[y]){
23                 LL t = lx[x]+ly[y]-g[x][y];
24                 if(t==0){
25                     pa[y]=x;
26                     if(!My[y]){augment(y);return;}
27                     vy[y]=1, q.push(My[y]);
28                 }else if(Sy[y]>t) pa[y]=x, Sy[y]=t;
29             }
30         }
31         LL cut = INF;
32         for(int y=1; y<=n; ++y)
33             if(!vy[y]&&cut>Sy[y]) cut=Sy[y];
34         for(int j=1; j<=n; ++j){
35             if(vx[j]) lx[j] -= cut;
36             if(vy[j]) ly[j] += cut;
37             else Sy[j] -= cut;
38         }
39         for(int y=1; y<=n; ++y){
40             if(!vy[y]&&Sy[y]==0){
41                 if(!My[y]){augment(y);return;}
42                 vy[y]=1, q.push(My[y]);
43             }
44         }
45     }
46 }
47 LL KM(){
48     memset(My,0,sizeof(int)*(n+1));
49     memset(Mx,0,sizeof(int)*(n+1));
50     memset(ly,0,sizeof(LL)*(n+1));
51     for(int x=1; x<=n; ++x){
52         lx[x] = -INF;
53         for(int y=1; y<=n; ++y)
54             lx[x] = max(lx[x],g[x][y]);
55     }
56     for(int x=1; x<=n; ++x) bfs(x);
57     LL ans = 0;
58     for(int y=1; y<=n; ++y) ans+=g[My[y]][y];

```

```

59 return ans;
60 }

```

3.8 MaximumClique

```

1 struct MaxClique{
2     static const int MAXN=105;
3     int N,ans;
4     int g[MAXN][MAXN], dp[MAXN], stk[MAXN][MAXN];
5     int sol[MAXN], tmp[MAXN]; //sol[0~ans-1]為答案
6     void init(int n){
7         N=n; //0-base
8         memset(g,0,sizeof(g));
9     }
10    void add_edge(int u,int v){
11        g[u][v]=g[v][u]=1;
12    }
13    int dfs(int ns,int dep){
14        if(!ns){
15            if(dep>ans){
16                ans=dep;
17                memcpy(sol,tmp,sizeof tmp);
18                return 1;
19            }else return 0;
20        }
21        for(int i=0; i<ns; ++i){
22            if(dep+ns-i<ans) return 0;
23            int u=stk[dep][i], cnt=0;
24            if(dep+dp[u]<=ans) return 0;
25            for(int j=i+1; j<ns; ++j){
26                int v=stk[dep][j];
27                if(g[u][v]) stk[dep+1][cnt++]=v;
28            }
29            tmp[dep]=u;
30            if(dfs(cnt,dep+1)) return 1;
31        }
32        return 0;
33    }
34    int clique(){
35        int u,v,ns;
36        for(ans=0,u=N-1;u>=0;--u){
37            for(ns=0,tmp[0]=u,v=u+1;v<N;v++){
38                if(g[u][v]) stk[1][ns++]=v;
39                dfs(ns,1), dp[u]=ans;
40            }
41            return ans;
42        }
43    };

```

3.9 MinimumMeanCycle

```

1 #include<cstdio> //for DBL_MAX
2 int dp[MAXN][MAXN]; // 1-base, O(NM)
3 vector<tuple<int,int,int>> edge;
4 double mmc(int n){ //allow negative weight
5     const int INF=0x3f3f3f3f;
6     for(int t=0; t<n; ++t){

```

```

7     memset(dp[t+1],0x3f,sizeof(dp[t+1]));
8     for(const auto &e:edge){
9         int u,v,w;
10        tie(u,v,w) = e;
11        dp[t+1][v]=min(dp[t+1][v],dp[t][u]+w);
12    }
13 }
14 double res = DBL_MAX;
15 for(int u=1; u<=n; ++u){
16     if(dp[n][u]==INF) continue;
17     double val = -DBL_MAX;
18     for(int t=0; t<n; ++t)
19         val=max(val,(dp[n][u]-dp[t][u])*1.0/(n-t));
20     res=min(res,val);
21 }
22 return res;
23 }

```

3.10 Rectilinear MST

```

1 //平面曼哈頓最小生成樹構造圖(去除非必要邊)
2 #define T int
3 #define INF 0x3f3f3f3f
4 struct point{
5     T x,y;
6     int id; //從0開始編號
7     point(){}
8     T dist(const point &p)const{
9         return abs(x-p.x)+abs(y-p.y);
10    };
11 };
12 bool cmpx(const point &a,const point &b){
13     return a.x<b.x||(a.x==b.x&&a.y<b.y);
14 }
15 struct edge{
16     int u,v;
17     T cost;
18     edge(int u,int v,T c):u(u),v(v),cost(c){}
19     bool operator<(const edge&e)const{
20         return cost<e.cost;
21     }
22 };
23 struct bit_node{
24     T mi;
25     int id;
26     bit_node(const T&mi=INF,int id=-1):mi(mi),id(id){}
27 };
28 vector<bit_node> bit;
29 void bit_update(int i,const T&data,int id){
30     for(;;i=i&(-i)){
31         if(data<bit[i].mi) bit[i]=bit_node(data,id);
32     }
33 }
34 int bit_find(int i,int m){
35     bit_node x;
36     for(;;i<=m;i+=i&(-i)) if(bit[i].mi<x.mi)x=bit[i];
37     return x.id;
38 }
39 vector<edge> build_graph(int n,point p[]){

```

```

40     vector<edge> e; //edge for MST
41     for(int dir=0; dir<4; ++dir){ //4種座標變換
42         if(dir%2) for(int i=0; i<n; ++i) swap(p[i].x,p[i].y);
43         else if(dir==2) for(int i=0; i<n; ++i) p[i].x=-p[i].x;
44         sort(p,p+n,cmpx);
45         vector<T> ga(n), gb;
46         for(int i=0; i<n; ++i) ga[i]=p[i].y-p[i].x;
47         gb=ga, sort(gb.begin(),gb.end());
48         gb.erase(unique(gb.begin(),gb.end()),gb.end());
49         int m=gb.size();
50         bit=vector<bit_node>(m+1);
51         for(int i=n-1; i>=0; --i){
52             int pos=lower_bound(gb.begin(),gb.end(),ga[i])-gb.begin()+1;
53             int ans=bit_find(pos,m);
54             if(~ans)e.push_back(edge(p[i].id,p[ans].id,p[i].dist(p[ans])));
55             bit_update(pos,p[i].x+p[i].y,i);
56         }
57     }
58     return e;
59 }

```

3.11 treeISO

```

1 const int MAXN=100005;
2 const long long X=12327,P=0xdefaced;
3 vector<int> g[MAXN];
4 bool vis[MAXN];
5 long long dfs(int u){ //hash ver
6     vis[u]=1;
7     vector<long long> tmp;
8     for(auto v:g[u]) if(!vis[v]) tmp.push_back(dfs(v));
9     if(tmp.empty()) return 177;
10    long long ret=4931;
11    sort(tmp.begin(),tmp.end());
12    for(auto v:tmp) ret=((ret*X)^v)%P;
13    return ret;
14 }
15 //-----
16 string dfs(int x,int p){
17     vector<string> c;
18     for(int y:g[x])
19         if(y!=p) c.emplace_back(dfs(y,x));
20     sort(c.begin(),c.end());
21     string ret("(");
22     for(auto &s:c) ret+=s;
23     ret+=")";
24     return ret;
25 }

```

3.12 一般圖最小權完美匹配

```

1 struct Graph {
2     // Minimum General Weighted Matching (
3     // Perfect Match) 0-base

```

```

3 static const int MXN = 105;
4 int n, edge[MXN][MXN];
5 int match[MXN], dis[MXN], onstk[MXN];
6 vector<int> stk;
7 void init(int _n) {
8     n = _n;
9     for (int i=0; i<n; i++)
10         for (int j=0; j<n; j++)
11             edge[i][j] = 0;
12 }
13 void add_edge(int u, int v, int w) {
14     edge[u][v] = edge[v][u] = w;
15 }
16 bool SPFA(int u) {
17     if (onstk[u]) return true;
18     stk.push_back(u);
19     onstk[u] = 1;
20     for (int v=0; v<n; v++) {
21         if (u != v && match[u] != v && !onstk[v]) {
22             int m = match[v];
23             if (dis[m] > dis[u] - edge[v][m] +
24                 edge[u][v]) {
25                 dis[m] = dis[u] - edge[v][m] +
26                     edge[u][v];
27                 onstk[v] = 1;
28                 stk.push_back(v);
29                 if (SPFA(m)) return true;
30                 stk.pop_back();
31                 onstk[v] = 0;
32             }
33         }
34     }
35     onstk[u] = 0;
36     stk.pop_back();
37     return false;
38 }
39 int solve() {
40     // find a match
41     for (int i=0; i<n; i+=2) {
42         match[i] = i+1, match[i+1] = i;
43     }
44     for(;;) {
45         int found = 0;
46         for (int i=0; i<n; i++) dis[i] = onstk[i] = 0;
47         for (int i=0; i<n; i++) {
48             stk.clear();
49             if (!onstk[i] && SPFA(i)) {
50                 found = 1;
51                 while (stk.size()>=2) {
52                     int u = stk.back(); stk.pop_back();
53                     int v = stk.back(); stk.pop_back();
54                     match[u] = v;
55                     match[v] = u;
56                 }
57                 if (!found) break;
58             }
59         }
60         int ret = 0;
61         for (int i=0; i<n; i++)
62             ret += edge[i][match[i]];
63         ret /= 2;

```

3.13 全局最小割

```

1 const int INF=0x3f3f3f3f;
2 template<typename T>
3 struct stoer_wagner{// 0-base
4     static const int MAXN=150;
5     T g[MAXN][MAXN], dis[MAXN];
6     int nd[MAXN], n, s, t;
7     void init(int _n) {
8         n=_n;
9         for (int i=0; i<n; i++)
10             for (int j=0; j<n; j++) g[i][j]=0;
11     }
12     void add_edge(int u, int v, T w) {
13         g[u][v]=g[v][u]+=w;
14     }
15     T min_cut() {
16         T ans=INF;
17         for (int i=0; i<n; i++) nd[i]=i;
18         for (int ind, tn=n; tn>1; --tn) {
19             for (int i=1; i<tn; i++) dis[ind[i]]=0;
20             for (int i=1; i<tn; i++) {
21                 ind=i;
22                 for (int j=i; j<tn; j++) {
23                     dis[ind[j]]+=g[ind[i-1]][nd[j]];
24                     if (dis[ind[ind]]<dis[ind[j]]) ind=j;
25                 }
26                 swap(nd[ind], nd[i]);
27             }
28             if (ans>dis[ind[ind]]) ans=dis[t=nd[ind]
29                 ], s=nd[ind-1];
30             for (int i=0; i<tn; i++)
31                 g[ind[ind-1]][nd[i]]=g[ind[i]][nd[ind]
32                     -1]+g[ind[i]][nd[ind]];
33         }
34         return ans;
35     }
36 }

```

3.14 弦圖完美消除序列

```

1 struct chordal {
2     static const int MAXN=1005;
3     int n; // 0-base
4     vector<int> G[MAXN];
5     int rank[MAXN], label[MAXN];
6     bool mark[MAXN];
7     void init(int _n) { n=_n;
8         for (int i=0; i<n; i++) G[i].clear();
9     }
10    void add_edge(int u, int v) {
11        G[u].push_back(v);
12        G[v].push_back(u);
13    }
14    vector<int> MCS() {
15        memset(rank, -1, sizeof(int)*n);

```

```

16    memset(label, 0, sizeof(int)*n);
17    priority_queue<pair<int, int>> pq;
18    for (int i=0; i<n; i++) pq.push(make_pair(0, i));
19    for (int i=n-1; i>=0; --i) for(;;) {
20        int u=pq.top().second; pq.pop();
21        if (~rank[u]) continue;
22        rank[u]=i;
23        for (auto v:G[u]) if (rank[v]==-1) {
24            pq.push(make_pair(++label[v], v));
25        }
26        break;
27    }
28    vector<int> res(n);
29    for (int i=0; i<n; i++) res[rank[i]]=i;
30    return res;
31 }
32 bool check(vector<int> ord) { // 弦圖判定
33     for (int i=0; i<n; i++) rank[ord[i]]=i;
34     memset(mark, 0, sizeof(bool)*n);
35     for (int i=0; i<n; i++) {
36         vector<pair<int, int>> tmp;
37         for (auto u:G[ord[i]]) if (!mark[u])
38             tmp.push_back(make_pair(rank[u], u));
39         sort(tmp.begin(), tmp.end());
40         if (tmp.size()) {
41             int u=tmp[0].second;
42             set<int> S;
43             for (auto v:G[u]) S.insert(v);
44             for (size_t j=1; j<tmp.size(); ++j)
45                 if (!S.count(tmp[j].second)) return
46                     0;
47             mark[ord[i]]=1;
48         }
49         return 1;
50     }
51 }

```

3.15 最小斯坦納樹 DP

```

1 // n個點，其中r個要構成斯坦納樹
2 // 答案在max(dp[(1<<r)-1][k]) k=0~n-1
3 // p表示要構成斯坦納樹的點集
4 // O((n^3 + n^3*r + n^2*2^r))
5 #define REP(i, n) for (int i=0; i<(int)n; ++i)
6 const int MAXN=30, MAXM=8; // 0-base
7 const int INF=0x3f3f3f3f;
8 int dp[1<<MAXM][MAXN];
9 int g[MAXN][MAXN]; // 圖
10 void init() { memset(g, 0x3f, sizeof(g)); }
11 void add_edge(int u, int v, int w) {
12     g[u][v]=g[v][u]=min(g[v][u], w);
13 }
14 void steiner(int n, int r, int *p) {
15     REP(k, n) REP(i, n) REP(j, n)
16         g[i][j]=min(g[i][j], g[i][k]+g[k][j]);
17     REP(i, n) g[i][i]=0;
18     REP(i, r) REP(j, n) dp[1<<i][j]=g[p[i]][j];
19     for (int i=1; i<(1<<r); ++i) {
20         if (!(i&(i-1))) continue;
21         REP(j, n) dp[i][j]=INF;

```

3.16 最小樹形圖朱劉

```

1 template<typename T>
2 struct zhu_liu {
3     static const int MAXN=110, MAXM=10005;
4     struct node {
5         int u, v;
6         T w, tag;
7         node *l, *r;
8         node(int u=0, int v=0, T w=0): u(u), v(v), w(w), tag(0), l(0), r(0) {}
9     };
10    void down() {
11        w+=tag;
12        if (l) l->tag+=tag;
13        if (r) r->tag+=tag;
14        tag=0;
15    }
16    mem[[MAXN]; // 靜態記憶體
17    node *pq[MAXN*2], *E[[MAXN*2];
18    int st[MAXN*2], id[MAXN*2], m;
19    void init(int n) {
20        for (int i=1; i<=n; ++i) {
21            pq[i]=E[i]=0, st[i]=id[i]=i;
22            m+=0;
23        }
24        node *merge(node *a, node *b) { // skew heap
25            if (!a || !b) return a? a: b;
26            a->down(), b->down();
27            if (b->w<a->w) return merge(b, a);
28            swap(a->l, a->r);
29            a->l=merge(b, a->l);
30            return a;
31        }
32        void add_edge(int u, int v, T w) {
33            if (u!=v) pq[v]=merge(pq[v], &(mem[m++]=node(u, v, w)));
34        }
35        int find(int x, int *st) {
36            return st[x]=x? x: st[x]=find(st[x], st);
37        }
38        T build(int root, int n) {
39            T ans=0; int N=n, all=n;
40            for (int i=1; i<=N; ++i) {
41                if (i==root || !pq[i]) continue;
42                while (pq[i]) {
43                    pq[i]->down(), E[i]=pq[i];
44                    pq[i]=merge(pq[i]->l, pq[i]->r);
45                    if (find(E[i]->u, id)!=find(i, id))
46                        break;
47                }
48            }
49            if (find(E[1]->u, id)!=find(1, id))
50                continue;
51            ans+=E[1]->w;

```

```

48 if(find(E[i]->u,st)==find(i,st)){
49     if(pq[i])pq[i]->tag-=E[i]->w;
50     pq[++N]=pq[i];id[N]=N;
51     for(int u=find(E[i]->u,id);u!=i;u=
        find(E[u]->u,id)){
52         if(pq[u])pq[u]->tag-=E[u]->w;
53         id[find(u,id)]=N;
54         pq[N]=merge(pq[N],pq[u]);
55     }
56     st[N]=find(i,st);
57     id[find(i,id)]=N;
58 }else st[find(i,st)]=find(E[i]->u,st)
    ,--all;
59 }
60 return all==1?ans:-INT_MAX;//圖不連通就
    無解
61 }
62 };

```

3.17 穩定婚姻模板

```

1 queue<int> Q;
2 for ( i : 所有考生 ) {
3     設定在第0志願;
4     Q.push( 考生i );
5 }
6 while(Q.size()){
7     當前考生=Q.front();Q.pop();
8     while ( 此考生未分發 ) {
9         指標移到下一志願;
10        if ( 已經沒有志願 or 超出志願總數 )
11            break;
12        計算該考生在該科系加權後的總分;
13        if ( 不符合科系需求 ) continue;
14        if ( 目前科系有餘額 ) {
15            依加權後分數高低順序將考生id加入科系錄
16            取名單中;
17            break;
18        }
19        if ( 目前科系已額滿 ) {
20            if ( 此考生成績比最低分數還高 ) {
21                依加權後分數高低順序將考生id加入科系
22                錄取名單;
23                Q.push( 被踢出的考生 );
24            }
25        }
26    }
27 }

```

4 Language

4.1 CNF

```

1 #define MAXN 55
2 struct CNF{

```

```

3     int s,x,y;//s->xy | s->x, if y==-1
4     int cost;
5     CNF(){}
6     CNF(int s,int x,int y,int c):s(s),x(x),y(y)
7     ,cost(c){}
8 };
9 int state;//規則數量
10 map<char,int> rule;//每個字元對應到的規則 ·
11     小寫字母為終端字符
12 vector<CNF> cnf;
13 void init(){
14     state=0;
15     rule.clear();
16     cnf.clear();
17 }
18 void add_to_cnf(char s,const string &p,int
19     cost){
20     //加入一個s -> <p>的文法 · 代價為cost
21     if(rule.find(s)==rule.end())rule[s]=state
22     ++;
23     for(auto c:p)if(rule.find(c)==rule.end())
24         rule[c]=state++;
25     if(p.size()==1){
26         cnf.push_back(CNF(rule[s],rule[p[0]],-1,
27             cost));
28     }else{
29         int left=rule[s];
30         int sz=p.size();
31         for(int i=0; i<sz-2; ++i){
32             cnf.push_back(CNF(left,rule[p[i]],
33                 state,0));
34             left=state++;
35         }
36         cnf.push_back(CNF(left,rule[p[sz-2]],
37             rule[p[sz-1]],cost));
38     }
39 }
40 vector<long long> dp[MAXN][MAXN];
41 vector<bool> neg_INF[MAXN][MAXN];//如果花費
42     是負的可能會有無限小的情形
43 void relax(int l,int r,const CNF &c,long
44     long cost,bool neg_c=0){
45     if(!neg_INF[l][r][c.s]&&(neg_INF[l][r][c.x]
46         ||cost<dp[l][r][c.s])){
47         if(neg_c||neg_INF[l][r][c.x]){
48             dp[l][r][c.s]=0;
49             neg_INF[l][r][c.s]=true;
50         }else dp[l][r][c.s]=cost;
51     }
52 }
53 void bellman(int l,int r,int n){
54     for(int k=1;k<=state;++k)
55         for(auto c:cnf)
56             if(c.y==-1)relax(l,r,c,dp[l][r][c.x]+
57                 .cost,k==n);
58 }
59 void cyk(const vector<int> &tok){
60     for(int i=0;i<(int)tok.size();++i){
61         for(int j=0;j<(int)tok.size();++j){
62             dp[i][j]=vector<long long>(state+1,
63                 INT_MAX);
64             neg_INF[i][j]=vector<bool>(state+1,
65                 false);
66         }
67     }
68     dp[i][i][tok[i]]=0;

```

```

54     bellman(i,i,tok.size());
55 }
56 for(int r=1;r<(int)tok.size();++r){
57     for(int l=r-1;l>=0;--l){
58         for(int k=1;k<=state;++k)
59             for(auto c:cnf)
60                 if(~c.y)relax(l,r,c,dp[l][k][c.x]+
61                     dp[k+1][r][c.y]+c.cost);
62         bellman(l,r,tok.size());
63     }
64 }

```

5 Linear Programming

5.1 simplex

```

1 /**target:
2     max \sum_{j=1}^n A_{0,j}*x_j
3 condition:
4     \sum_{j=1}^n A_{i,j}*x_j <= A_{i,0} | i=1~m
5     x_j >= 0 | j=1~n
6 VDB = vector<double>*/
7 template<class VDB>
8 VDB simplex(int m,int n,vector<VDB> a){
9     vector<int> left(m+1), up(n+1);
10     iota(left.begin(), left.end(), n);
11     iota(up.begin(), up.end(), 0);
12     auto pivot = [&](int x, int y){
13         swap(left[x], up[y]);
14         auto k = a[x][y]; a[x][y] = 1;
15         vector<int> pos;
16         for(int j = 0; j <= n; ++j){
17             a[x][j] /= k;
18             if(a[x][j] != 0) pos.push_back(j);
19         }
20         for(int i = 0; i <= m; ++i){
21             if(a[i][y]==0 || i == x) continue;
22             k = a[i][y], a[i][y] = 0;
23             for(int j : pos) a[i][j] -= k*a[x][j];
24         }
25     };
26     for(int x,y;;){
27         for(int i=x+1; i <= m; ++i)
28             if(a[i][0]<a[x][0]) x = i;
29         if(a[x][0]>=0) break;
30         for(int j=y+1; j <= n; ++j)
31             if(a[x][j]<a[x][y]) y = j;
32         if(a[x][y]>=0) return VDB();//infeasible
33         pivot(x, y);
34     }
35     for(int x,y;;){
36         for(int j=y+1; j <= n; ++j)
37             if(a[0][j] > a[0][y]) y = j;
38         if(a[0][y]<=0) break;
39         x = -1;
40         for(int i=1; i<=m; ++i) if(a[i][y] > 0)
41             if(x == -1 || a[i][0]/a[i][y]
42                 < a[x][0]/a[x][y]) x = i;
43         if(x == -1) return VDB();//unbounded
44         pivot(x, y);

```

6 Number Theory

6.1 basic

```

1 template<typename T>
2 void gcd(const T &a,const T &b,T &d,T &x,T &y){
3     if(!b) d=a,x=1,y=0;
4     else gcd(b,a%b,d,y,x), y-=x*(a/b);
5 }
6 long long int phi[N+1];
7 void phiTable(){
8     for(int i=1;i<=N;i++)phi[i]=i;
9     for(int i=1;i<=N;i++)for(x=i*2;x<=N;x+=i)
10         phi[x]-=phi[i];
11 }
12 void all_divdown(const LL &n) { // all n/x
13     for(LL a=1;a<=n;a=n/(n/(a+1))) {
14         // dosomething;
15     }
16 }
17 const int MAXPRIME = 1000000;
18 int iscom[MAXPRIME], prime[MAXPRIME],
19     primecnt;
20 int phi[MAXPRIME], mu[MAXPRIME];
21 void sieve(void){
22     memset(iscom,0,sizeof(iscom));
23     primecnt = 0;
24     phi[1] = mu[1] = 1;
25     for(int i=2;i<MAXPRIME;++i) {
26         if(!iscom[i]) {
27             prime[primecnt++] = i;
28             mu[i] = -1;
29             phi[i] = i-1;
30         }
31         for(int j=0;j<primecnt;++j) {
32             int k = i * prime[j];
33             if(k>MAXPRIME) break;
34             iscom[k] = prime[j];
35             if(i%prime[j]==0) {
36                 mu[k] = 0;
37                 phi[k] = phi[i] * prime[j];
38                 break;
39             } else {
40                 mu[k] = -mu[i];
41                 phi[k] = phi[i] * (prime[j]-1);
42             }
43         }
44     }
45 }
46 bool g_test(const LL &g, const LL &p, const
47     vector<LL> &v) {

```

```

46 for(int i=0;i<v.size();++i)
47     if(modexp(g,(p-1)/v[i],p)==1)
48         return false;
49 return true;
50 }
51 LL primitive_root(const LL &p) {
52     if(p==2) return 1;
53     vector<LL> v;
54     Factor(p-1,v);
55     v.erase(unique(v.begin(), v.end()), v.end()
56         ());
57     for(LL g=2;g<p;++g)
58         if(g_test(g,p,v))
59             puts("primitive_root NOT FOUND");
60     return -1;
61 }
62 int Legendre(const LL &a, const LL &p) {
63     return modexp(a%p,(p-1)/2,p); }
64 LL inv(const LL &a, const LL &n) {
65     LL d,x,y;
66     gcd(a,n,d,x,y);
67     return d==1 ? (x+n)%n : -1;
68 }
69 int inv[maxN];
70 LL invtable(int n,LL P){
71     inv[1]=1;
72     for(int i=2;i<n;++i)
73         inv[i]=(P-(P/i))*inv[P%i]%P;
74 }
75 LL log_mod(const LL &a, const LL &b, const
76     LL &p) {
77     // a ^ x = b ( mod p )
78     int m=sqrt(p+.5), e=1;
79     LL v=inv(modexp(a,m,p), p);
80     map<LL,int> x;
81     x[1]=0;
82     for(int i=1;i<m;++i) {
83         e = LLMul(e,a,p);
84         if(!x.count(e)) x[e] = i;
85     }
86     for(int i=0;i<m;++i) {
87         if(x.count(b)) return i*m + x[b];
88         b = LLMul(b,v,p);
89     }
90     return -1;
91 }
92 LL Tonelli_Shanks(const LL &n, const LL &p)
93 {
94     // x^2 = n ( mod p )
95     if(n==0) return 0;
96     if(Legendre(n,p)!=1) while(1) { puts("SQRT
97         ROOT does not exist"); }
98     int S = 0;
99     LL Q = p-1;
100     while( !(Q&1) ) { Q>>=1; ++S; }
101     if(S==1) return modexp(n%p,(p+1)/4,p);
102     LL z = 2;
103     for(; Legendre(z,p)!=-1;++z)
104         LL c = modexp(z,Q,p);
105         LL R = modexp(n%p,(Q+1)/2,p), t = modexp(n
106             %p,Q,p);

```

```

106 int M = S;
107 while(1) {
108     if(t==1) return R;
109     LL b = modexp(c,1L<<(M-i-1),p);
110     R = LLMul(R,b,p);
111     t = LLMul( LLMul(b,b,p), t, p);
112     c = LLMul(b,b,p);
113     M = i;
114 }
115 return -1;
116 }
117 template<typename T>
118 T Euler(T n){
119     T ans=n;
120     for(T i=2;i*i<=n;++i){
121         if(n%i==0){
122             ans=ans/i*(i-1);
123             while(n%i==0)n/=i;
124         }
125     }
126     if(n>1)ans=ans/n*(n-1);
127     return ans;
128 }
129 //Chinese_remainder_theorem
130 template<typename T>
131 T pow_mod(T n,T k,T m){
132     T ans=1;
133     for(n=(n>m?n%m:n);k;k>>=1){
134         if(k&1)ans=ans*n%m;
135         n=n*n%m;
136     }
137     return ans;
138 }
139 template<typename T>
140 T crt(vector<T> &m,vector<T> &a){
141     T M=1,tM,ans=0;
142     for(int i=0;i<(int)m.size();++i)M*=m[i];
143     for(int i=0;i<(int)a.size();++i){
144         tM=M/m[i];
145         ans=(ans+(a[i]*tM%M)*pow_mod(tM,Euler(m[
146             i])-1,m[i])%M)%M;
147     }
148     /*如果m[i]是質數，Euler(m[i])-1=m[i]-2，
149     就不用算Euler了*/
150     return ans;
151 }
152 //java code
153 //求sqrt(N)的連分數
154 public static void Pell(int n){
155     BigInteger N,p1,p2,q1,q2,a0,a1,a2,g1,g2,h1
156         ,h2,p,q;
157     g1=q2=p1=BigInteger.ZERO;
158     h1=q1=p2=BigInteger.ONE;
159     a0=a1=BigInteger.valueOf((int)Math.sqrt
160         (1.0*n));
161     BigInteger ans=a0.multiply(a0);
162     if(ans.equals(BigInteger.valueOf(n))){
163         System.out.println("No solution!");
164         return ;
165     }
166     while(true){
167         g2=a1.multiply(h1).subtract(g1);

```

```

167     h2=N.subtract(g2.pow(2)).divide(h1);
168     a2=g2.add(a0).divide(h2);
169     p=a1.multiply(p2).add(p1);
170     q=a1.multiply(q2).add(q1);
171     if(p.pow(2).subtract(N.multiply(q.pow
172         (2))).compareTo(BigInteger.ONE)==0)
173         break;
174     g1=g2;h1=h2;a1=a2;
175     p1=p2;p2=p;
176     q1=q2;q2=q;
177 }
178 System.out.println(p+" "+q);

```

6.2 bit set

```

1 void sub_set(int S){
2     int sub=S;
3     do{
4         //對某集合的子集的處理
5         sub=(sub-1)&S;
6     }while(sub!=S);
7 }
8 void k_sub_set(int k,int n){
9     int comb=(1<<k)-1,S=1<<n;
10    while(comb<S){
11        //對大小為k的子集的處理
12        int x=comb&-comb,y=comb+x;
13        comb=((comb~y)/x>>1)|y;
14    }
15 }

```

6.3 cantor expansion

```

1 int factorial[MAXN];
2 void init(){
3     factorial[0]=1;
4     for(int i=1;i<=MAXN;++i)factorial[i]=
5         factorial[i-1]*i;
6 }
7 int encode(const vector<int> &s){
8     int n=s.size(),res=0;
9     for(int i=0;i<n;++i){
10        int t=0;
11        for(int j=i+1;j<n;++j)
12            if(s[j]<s[i])++t;
13        res+=t*factorial[n-i-1];
14    }
15    return res;
16 }
17 vector<int> decode(int a,int n){
18     vector<int> res;
19     vector<bool> vis(n,0);
20     for(int i=n-1;i>=0;--i){
21         int t=a/factorial[i],j;
22         for(j=0;j<n;++j)
23             if(!vis[j]){
24                 if(t==0)break;
25                 --t;

```

```

25     }
26     res.push_back(j);
27     vis[j]=1;
28     a%=factorial[i];
29 }
30 return res;
31 }

```

6.4 FFT

```

1 template<typename T,typename VT=vector<
2     complex<T> > >
3 struct FFT{
4     const T pi;
5     FFT(const T pi=acos((T)-1)):pi(pi){}
6     unsigned bit_reverse(unsigned a,int len){
7         a=((a&0x55555555U)<<1)|((a&0xAAAAAAAAU)>>1);
8         a=((a&0x33333333U)<<2)|((a&0xCCCCCCCCU)>>2);
9         a=((a&0x0F0F0F0FU)<<4)|((a&0xFF0F0F0FU)>>4);
10        a=((a&0x00FF00FFU)<<8)|((a&0xFFFF0000U)
11            >>16);
12        return a>>(32-len);
13    }
14    void fft(bool is_inv,VT &in,VT &out,int N)
15    {
16        int bitlen=__lg(N),num=is_inv?-1:1;
17        for(int i=0;i<N;++i)out[bit_reverse(i,
18            bitlen)]=in[i];
19        for(int step=2;step<=N;step<=1){
20            const int mh=step>>1;
21            for(int i=0;i<mh;++i){
22                complex<T> wi=exp(complex<T>(0,i*num
23                    *pi/mh));
24                for(int j=i;j<N;j+=step){
25                    int k=j+mh;
26                    complex<T> u=out[j],t=wi*out[k];
27                    out[j]=u+t;
28                    out[k]=u-t;
29                }
30            }
31        }
32    }
33 };

```

6.5 find real root

```

1 // an*x^n + ... + a1x + a0 = 0;
2 int sign(double x){
3     return x < -eps ? -1 : x > eps;
4 }
5 double get(const vector<double>&coef, double
6     x){
7     double e = 1, s = 0;
8     for(auto i : coef) s += i*e, e *= x;
9     return s;
10 }
11

```

```

12 double find(const vector<double>&coef, int n
13     , double lo, double hi){
14     double sign_lo, sign_hi;
15     if( !(sign_lo = sign(get(coef,lo))) )
16         return lo;
17     if( !(sign_hi = sign(get(coef,hi))) )
18         return hi;
19     if(sign_lo * sign_hi > 0) return INF;
20     for(int stp = 0; stp < 100 && hi - lo >
21         eps; ++stp){
22         double m = (lo+hi)/2.0;
23         int sign_mid = sign(get(coef,m));
24         if(!sign_mid) return m;
25         if(sign_lo*sign_mid < 0) hi = m;
26         else lo = m;
27     }
28     return (lo+hi)/2.0;
29 }
30 vector<double> cal(vector<double>coef, int n
31 ){
32     vector<double>res;
33     if(n == 1){
34         if(sign(coef[1])) res.pb(-coef[0]/coef
35             [1]);
36         return res;
37     }
38     vector<double>dcoef(n);
39     for(int i = 0; i < n; ++i) dcoef[i] = coef
40         [i+1]*(i+1);
41     vector<double>droot = cal(dcoef, n-1);
42     droot.insert(droot.begin(), -INF);
43     droot.pb(INF);
44     for(int i = 0; i+1 < droot.size(); ++i){
45         double tmp = find(coef, n, droot[i],
46             droot[i+1]);
47         if(tmp < INF) res.pb(tmp);
48     }
49     return res;
50 }
51 int main () {
52     vector<double>ve;
53     vector<double>ans = cal(ve, n);
54     // 視情況把答案 +eps · 避免 -0
55 }

```

6.6 FWT

```

1 vector<int> F_OR_T(vector<int> f, bool
2     inverse){
3     for(int i=0; (2<<i)<=f.size(); ++i)
4         for(int j=0; j<f.size(); j+=2<<i)
5             for(int k=0; k<(1<<i); ++k)
6                 f[j+k+(1<<i)] += f[j+k]*(inverse
7                     ?-1:1);
8     return f;
9 }
10 vector<int> rev(vector<int> A) {
11     for(int i=0; i<A.size(); i+=2)
12         swap(A[i],A[i^(A.size()-1)]);
13     return A;
14 }

```

```

13 vector<int> F_AND_T(vector<int> f, bool
14     inverse){
15     return rev(F_OR_T(rev(f), inverse));
16 }
17 vector<int> F_XOR_T(vector<int> f, bool
18     inverse){
19     for(int i=0; (2<<i)<=f.size(); ++i)
20         for(int j=0; j<f.size(); j+=2<<i)
21             for(int k=0; k<(1<<i); ++k){
22                 int u=f[j+k], v=f[j+k+(1<<i)];
23                 f[j+k+(1<<i)] = u-v, f[j+k] = u+v;
24             }
25     if(inverse) for(auto &a:f) a/=f.size();
26     return f;
27 }

```

6.7 LinearCongruence

```

1 pair<LL,LL> LinearCongruence(LL a[],LL b[],
2     LL m[],int n) {
3     // a[i]*x = b[i] ( mod m[i] )
4     for(int i=0;i<n;++i) {
5         LL x, y, d = extgcd(a[i],m[i],x,y);
6         if(b[i]%d!=0) return make_pair(-1LL,0LL);
7         m[i] /= d;
8         b[i] = LLmul(b[i]/d,x,m[i]);
9     }
10    LL lastb = b[0], lastm = m[0];
11    for(int i=1;i<n;++i) {
12        LL x, y, d = extgcd(m[i],lastm,x,y);
13        if((lastb-b[i])%d!=0) return make_pair
14            (-1LL,0LL);
15        lastb = LLmul((lastb-b[i])/d,x,(lastm/d)
16            )*m[i];
17        lastm = (lastm/d)*m[i];
18        lastb = (lastb+b[i])%lastm;
19    }
20    return make_pair(lastb<0?lastb+lastm:lastb
21        ,lastm);
22 }

```

6.8 Lucas

```

1 ll C(ll n, ll m, ll p){ // n!/m!/(n-m)!
2     if(n<m) return 0;
3     return f[n]*inv(f[m],p)*p*inv(f[n-m],p)%p;
4 }
5 ll L(ll n, ll m, ll p){
6     if(!m) return 1;
7     return C(n%p,m%p,p)*L(n/p,m/p,p)%p;
8 }
9 ll Wilson(ll n, ll p){ // n!%p
10    if(!n)return 1;
11    ll res=Wilson(n/p, p);
12    if((n/p)%2) return res*(p-f[n%p])%p;
13    return res*f[n%p]%p; //(p-1)!%p=-1
14 }

```

6.9 Matrix

```

1 template<typename T>
2 struct Matrix{
3     using rt = std::vector<T>;
4     using mt = std::vector<rt>;
5     using matrix = Matrix<T>;
6     int r,c;
7     mt m;
8     Matrix(int r,int c):r(r),c(c),m(r,rt(c)){
9         rt& operator[](int i){return m[i];}
10        matrix operator+(const matrix &a){
11            matrix rev(r,c);
12            for(int i=0;i<r;++i)
13                for(int j=0;j<c;++j)
14                    rev[i][j]=m[i][j]+a.m[i][j];
15            return rev;
16        }
17        matrix operator-(const matrix &a){
18            matrix rev(r,c);
19            for(int i=0;i<r;++i)
20                for(int j=0;j<c;++j)
21                    rev[i][j]=m[i][j]-a.m[i][j];
22            return rev;
23        }
24        matrix operator*(const matrix &a){
25            matrix rev(r,a.c);
26            matrix tmp(a.c,a.r);
27            for(int i=0;i<a.r;++i)
28                for(int j=0;j<a.c;++j)
29                    tmp[j][i]=a.m[i][j];
30            for(int i=0;i<r;++i)
31                for(int j=0;j<a.c;++j)
32                    for(int k=0;k<c;++k)
33                        rev.m[i][j]+=m[i][k]*tmp[j][k];
34            return rev;
35        }
36        bool inverse(){
37            Matrix t(r,r+c);
38            for(int y=0;y<r;y++){
39                t.m[y][c+y] = 1;
40                for(int x=0;x<c;++x)
41                    t.m[y][x]=m[y][x];
42            }
43            if( !t.gas() )
44                return false;
45            for(int y=0;y<r;y++){
46                for(int x=0;x<c;++x)
47                    m[y][x]=t.m[y][c+x]/t.m[y][y];
48            }
49            return true;
50        }
51        T gas(){
52            vector<T> lazy(r,1);
53            bool sign=false;
54            for(int i=0;i<r;++i){
55                if( m[i][i]==0 ) {
56                    int j=i+1;
57                    while(j<r&&!m[j][i])j++;
58                    if(j==r)continue;
59                    m[i].swap(m[j]);
60                    sign=!sign;
61                }
62                for(int j=0;j<r;j++){
63                    if(i==j)continue;
64                    lazy[j]=lazy[j]*m[i][i];
65                }
66            }
67            if(sign) lazy[0]=-lazy[0];
68            return lazy[0];
69        }
70    };

```

```

64 T mx=m[j][i];
65 for(int k=0;k<c;++k)
66     m[j][k]=m[j][k]*m[i][i]-m[i][k]*mx;
67 }
68 }
69 T det=sign?-1:1;
70 for(int i=0;i<r;++i){
71     det = det*m[i][i];
72     det = det/lazy[i];
73     for(auto &j:m[i])j/=lazy[i];
74 }
75 return det;
76 }
77 };

```

6.10 MillerRobin

```

1 ULL LLmul(ULL a, ULL b, const ULL &mod) {
2     LL ans=0;
3     while(b) {
4         if(b&1) {
5             ans+=a;
6             if(ans>=mod) ans-=mod;
7         }
8         a<<=1, b>>=1;
9         if(a>=mod) a-=mod;
10    }
11    return ans;
12 }
13 ULL mod_mul(ULL a,ULL b,ULL m){
14     a%=m,b%=m; /* fast for m < 2^58 */
15     ULL y=(ULL)((double)a*b/m+0.5);
16     ULL r=(a*b-y*m)%m;
17     return r<0?r+m:r;
18 }
19 template<typename T>
20 T pow(T a,T b,T mod){ // a^b%mod
21     T ans=1;
22     for(;b;a=mod_mul(a,a,mod),b>>=1)
23         if(b&1)ans=mod_mul(ans,a,mod);
24     return ans;
25 }
26 int sprp[3]={2,7,61}; //int範圍可解
27 int llsprr
28     [7]={2,325,9375,28178,450775,9780504,
29     1795265022}; //至少 unsigned long long範圍
30 template<typename T>
31 bool isprime(T n,int *sprp,int num){
32     if(n==2)return 1;
33     if(n<2||n%2==0)return 0;
34     int t=0;
35     T u=n-1;
36     for(;u%2==0;++t)u>>=1;
37     for(int i=0;i<num;++i){
38         T a=sprp[i]%n;
39         if(a==0||a==1||a==n-1)continue;
40         T x=pow(a,u,n);
41         if(x==1||x==n-1)continue;
42         for(int j=0;j<t;j++){
43             x=mod_mul(x,x,n);
44             if(x==1)return 0;
45         }
46     }
47     return 1;
48 }

```

```

44     if(x==n-1)break;
45 }
46 if(x==n-1)continue;
47 return 0;
48 }
49 return 1;
50 }

```

6.11 NTT

```

1 2615053605667*(2^18)+1,3
2 15*(2^27)+1,31
3 479*(2^21)+1,3
4 7*17*(2^23)+1,3
5 3*3*211*(2^19)+1,5
6 25*(2^22)+1,3
7 template<typename T, typename VT=vector<T> >
8 struct NTT{
9     const T P,G;
10     NTT(T p=(1<<23)*7*17+1,T g=3):P(p),G(g){}
11     unsigned bit_reverse(unsigned a,int len){
12         //Look FFT.cpp
13     }
14     T pow_mod(T n,T k,T m){
15         T ans=1;
16         for(n=(n>=m?n%m:n);k>=1){
17             if(k&1)ans=ans*n%m;
18             n=n*n%m;
19         }
20         return ans;
21     }
22     void ntt(bool is_inv,VT &in,VT &out,int N)
23     {
24         int bitlen=__lg(N);
25         for(int i=0;i<N;++i)out[bit_reverse(i,
26             bitlen)]=in[i];
27         for(int step=2,id=1;step<=N;step<=1,++
28             id){
29             T wn=pow_mod(G,(P-1)>>id,P),wi=1,u,t;
30             const int mh=step>>1;
31             for(int i=0;i<mh;++i){
32                 for(int j=i;j<N;j+=step){
33                     u=out[j],t=wi*out[j+mh]%P;
34                     out[j]=u+t;
35                     out[j+mh]=u-t;
36                     if(out[j]>=P)out[j]-=P;
37                     if(out[j+mh]<0)out[j+mh]+=P;
38                 }
39                 wi=wi*wn%P;
40             }
41             if(is_inv){
42                 for(int i=1;i<N/2;++i)swap(out[i],out[
43                     N-i]);
44                 T invn=pow_mod(N,P-2,P);
45                 for(int i=0;i<N;++i)out[i]=out[i]*invn
46                     %P;
47             }
48         }
49     };

```

6.12 Simpson

```

1 double simpson(double a,double b){
2     double c=a+(b-a)/2;
3     return (F(a)+4*F(c)+F(b))*(b-a)/6;
4 }
5 double asr(double a,double b,double eps,
6     double A){
7     double c=a+(b-a)/2;
8     double L=simpson(a,c),R=simpson(c,b);
9     if( abs(L+R-A)<15*eps )
10         return L+R+(L+R-A)/15.0;
11     return asr(a,c,eps/2,L)+asr(c,b,eps/2,R);
12 }
13 double asr(double a,double b,double eps){
14     return asr(a,b,eps,simpson(a,b));
15 }

```

6.13 外星模運算

```

1 //a[0]^a[1]^a[2]^...
2 #define maxn 1000000
3 int euler[maxn+5];
4 bool is_prime[maxn+5];
5 void init_euler(){
6     is_prime[1]=1; //不是質數
7     for(int i=1;i<=maxn;i++)euler[i]=i;
8     for(int i=2;i<=maxn;i++){
9         if(!is_prime[i]){ //是質數
10             euler[i]--;
11             for(int j=i<1;j<=maxn;j+=i){
12                 is_prime[j]=1;
13                 euler[j]=euler[j]/i*(i-1);
14             }
15         }
16     }
17 }
18 LL pow(LL a,LL b,LL mod){ //a^b%mod
19     LL ans=1;
20     for(;b;a=a*a%mod,b>>=1)
21         if(b&1)ans=ans*a%mod;
22     return ans;
23 }
24 bool isless(LL *a,int n,int k){
25     if(*a==1)return k>1;
26     if(--n==0)return *a<k;
27     int next=0;
28     for(LL b=1;b<k;next++)
29         b*=a;
30     return isless(a+1,n,next);
31 }
32 LL high_pow(LL *a,int n,LL mod){
33     if(*a==1||--n==0)return *a%mod;
34     int k=0,r=euler[mod];
35     for(LL tma=1;tma!=pow(*a,k+r,mod);++k)
36         tma=tma*(a%mod);
37     if(isless(a+1,n,k))return pow(*a,high_pow(
38         a+1,n,k),mod);
39     int tmd=high_pow(a+1,n,r), t=(tmd-k+r)%r;
40     return pow(*a,k+t,mod);
41 }
42 LL a[1000005];

```

```

42 int t,mod;
43 int main(){
44     init_euler();
45     scanf("%d",&t);
46     #define n 4
47     while(t--){
48         for(int i=0;i<n;++i)scanf("%Lld",&a[i]);
49         scanf("%d",&mod);
50         printf("%Lld\n",high_pow(a,n,mod));
51     }
52     return 0;
53 }

```

6.14 數位統計

```

1 ll d[65], dp[65][2]; //up區間是不是完整
2 ll dfs(int p,bool is8,bool up){
3     if(!p)return 1; // 回傳0是不是答案
4     if(!up&&~dp[p][is8])return dp[p][is8];
5     int mx = up?d[p]:9; //可以用的有那些
6     ll ans=0;
7     for(int i=0;i<mx;++i){
8         if( is8&&i==7 )continue;
9         ans += dfs(p-1,i==8,up&&i==mx);
10    }
11    if(!up)dp[p][is8]=ans;
12    return ans;
13 }
14 f(ll N){
15     int k=0;
16     while(N){ // 把數字先分解到陣列
17         d[++k] = N%10;
18         N/=10;
19     }
20     return dfs(k,false,true);
21 }

```

6.15 質因數分解

```

1 LL func(const LL n,const LL mod,const int c)
2 {
3     return (LLmul(n,n,mod)+c+mod)%mod;
4 }
5 LL pollrho(const LL n, const int c) { //循
6     環節長度
7     LL a=1, b=1;
8     a=func(a,n,c)%n;
9     b=func(b,n,c)%n; b=func(b,n,c)%n;
10    while(gcd(abs(a-b),n)==1) {
11        a=func(a,n,c)%n;
12        b=func(b,n,c)%n; b=func(b,n,c)%n;
13    }
14    return gcd(abs(a-b),n);
15 }
16 void prefactor(LL &n, vector<LL> &v) {
17     for(int i=0;i<12;++i) {
18         while(n%prime[i]==0) {

```

```

19         v.push_back(prime[i]);
20         n/=prime[i];
21     }
22 }
23 }
24 }
25 void smallfactor(LL n, vector<LL> &v) {
26     if(n<MAXPRIME) {
27         while(isp[(int)n]) {
28             v.push_back(isp[(int)n]);
29             n/=isp[(int)n];
30         }
31         v.push_back(n);
32     } else {
33         for(int i=0;i<primecnt&&prime[i]*prime[i]
34             ]<=n;++i) {
35             while(n%prime[i]==0) {
36                 v.push_back(prime[i]);
37                 n/=prime[i];
38             }
39             if(n!=1) v.push_back(n);
40         }
41     }
42 }
43 void comfactor(const LL &n, vector<LL> &v) {
44     if(n<1e9) {
45         smallfactor(n,v);
46         return;
47     }
48     if(Isprime(n)) {
49         v.push_back(n);
50         return;
51     }
52     LL d;
53     for(int c=3; c<=n; ++c) {
54         d = pollrho(n,c);
55         if(d!=n) break;
56     }
57     comfactor(d,v);
58     comfactor(n/d,v);
59 }
60 }
61 void Factor(const LL &x, vector<LL> &v) {
62     LL n = x;
63     if(n==1) { puts("Factor 1"); return; }
64     prefactor(n,v);
65     if(n==1) return;
66     comfactor(n,v);
67     sort(v.begin(),v.end());
68 }
69 }
70 void AllFactor(const LL &n,vector<LL> &v) {
71     vector<LL> tmp;
72     Factor(n,tmp);
73     v.clear();
74     v.push_back(1);
75     int len;
76     LL now=1;
77     for(int i=0;i<tmp.size();++i) {
78         if(i==0 || tmp[i]!=tmp[i-1]) {
79             len = v.size();
80             now = 1;
81         }
82         now*=tmp[i];
83         for(int j=0;j<len;++j)

```

```

84     v.push_back(v[j]*now);
85 }
86 }

```

7 String

7.1 AC 自動機

```

1  template<char L='a',char R='z'>
2  class ac_automaton{
3  public:
4      struct joe{
5          int next[R-L+1],fail,efl,ed,cnt_dp,vis;
6          joe():ed(0),cnt_dp(0),vis(0){
7              for(int i=0;i<=R-L;++i)next[i]=0;
8          };
9      public:
10         std::vector<joe> S;
11         std::vector<int> q;
12         int qs,qe,vt;
13         ac_automaton():S(1),qs(0),qe(0),vt(0){
14             void clear(){
15                 q.clear();
16                 S.resize(1);
17                 for(int i=0;i<=R-L;++i)S[0].next[i]=0;
18                 S[0].cnt_dp=S[0].vis=qs=qe=vt=0;
19             }
20             void insert(const char *s){
21                 int o=0;
22                 for(int i=0,id;s[i];++i){
23                     id=s[i]-L;
24                     if(!S[o].next[id]){
25                         S.push_back(joe());
26                         S[o].next[id]=S.size()-1;
27                     }
28                     o=S[o].next[id];
29                     ++S[o].ed;
30                 }
31             }
32             void build_fail(){
33                 S[0].fail=S[0].efl=-1;
34                 q.clear();
35                 q.push_back(0);
36                 ++qe;
37                 while(qs!=qe){
38                     int pa=q[qs++],id,t;
39                     for(int i=0;i<=R-L;++i){
40                         t=S[pa].next[i];
41                         if(!t)continue;
42                         id=S[pa].fail;
43                         while(~id&&!S[id].next[i])id=S[id].fail;
44                         S[t].fail=~id?S[id].next[i]:0;
45                         S[t].efl=S[S[t].fail].ed?S[t].fail:S[S[t].fail].efl;
46                         q.push_back(t);
47                         ++qe;
48                     }
49                 }
50             }

```

```

51         /*DP出每個前綴在字串s出現的次數並傳回所有
52         字串被s匹配成功的次數O(N*M)*/
53         int match_0(const char *s){
54             int ans=0,id,p=0,i;
55             for(i=0;s[i];++i){
56                 id=s[i]-L;
57                 while(!S[p].next[id]&&p=p[S[p].fail];
58                 if(!S[p].next[id])continue;
59                 p=S[p].next[id];
60                 ++S[p].cnt_dp; /*匹配成功則它所有後綴都
61                 可以被匹配(DP計算)*/
62             }
63             for(i=qe-1;i>=0;--i){
64                 ans+=S[q[i]].cnt_dp*S[q[i]].ed;
65                 if(~S[q[i]].fail)S[S[q[i]].fail].cnt_dp+=S[q[i]].cnt_dp;
66             }
67             return ans;
68         }
69         /*多串匹配走efl邊並傳回所有字串被s匹配成功
70         的次數O(N*M^1.5)*/
71         int match_1(const char *s)const{
72             int ans=0,id,p=0,t;
73             for(int i=0;s[i];++i){
74                 id=s[i]-L;
75                 while(!S[p].next[id]&&p=p[S[p].fail];
76                 if(!S[p].next[id])continue;
77                 p=S[p].next[id];
78                 if(S[p].ed)ans+=S[p].ed;
79                 if(t=S[p].efl;~t;t=S[t].efl){
80                     ans+=S[t].ed; /*因為都走efl邊所以保證
81                     匹配成功*/
82                 }
83             }
84             return ans;
85         }
86         /*枚舉(s的子字串a)的所有相異字串各恰一次
87         並傳回次數O(N*M^(1/3))*/
88         int match_2(const char *s){
89             int ans=0,id,p=0,t;
90             ++vt;
91             /*把戳記vt+=1，只要vt沒溢位，所有S[p].vis==vt就會變成false
92             這種利用vt的方法可以O(1)歸零vis陣列*/
93             for(int i=0;s[i];++i){
94                 id=s[i]-L;
95                 while(!S[p].next[id]&&p=p[S[p].fail];
96                 if(!S[p].next[id])continue;
97                 p=S[p].next[id];
98                 if(S[p].ed&&S[p].vis!=vt){
99                     S[p].vis=vt;
100                     ans+=S[p].ed;
101                 }
102                 for(t=S[p].efl;~t&&S[t].vis!=vt;t=S[t].efl){
103                     S[t].vis=vt;
104                     ans+=S[t].ed; /*因為都走efl邊所以保證
105                     匹配成功*/
106                 }
107             }
108             return ans;
109         }
110         /*把AC自動機變成真的自動機*/

```

```

105 void evolution(){
106     for(qs=1;qs!=qe;){
107         int p=q[qs++];
108         for(int i=0;i<=R-L;++i)
109             if(S[p].next[i]==0)S[p].next[i]=S[S[p].fail].next[i];
110     }
111 }
112 }

```

7.2 hash

```

1  #define MAXN 1000000
2  #define mod 1073676287
3  /*mod 必須要是質數*/
4  typedef long long T;
5  char s[MAXN+5];
6  T h[MAXN+5]; /*hash陣列*/
7  T h_base[MAXN+5]; /*h_base[n]=(prime^n)%mod*/
8  void hash_init(int len,T prime){
9      h_base[0]=1;
10     for(int i=1;i<=len;++i){
11         h[i]=(h[i-1]*prime+s[i-1])%mod;
12         h_base[i]=(h_base[i-1]*prime)%mod;
13     }
14 }
15 T get_hash(int l,int r){ /*閉區間寫法，設編號
16     為0 ~ Len-1*/
17     return (h[r+1]-(h[l]*h_base[r-l+1])%mod+mod)%mod;

```

7.3 KMP

```

1  /*產生fail function*/
2  void kmp_fail(char *s,int len,int *fail){
3      int id=-1;
4      fail[0]=-1;
5      for(int i=1;i<len;++i){
6          while(~id&&s[id+1]!=s[i])id=fail[id];
7          if(s[id+1]==s[i])++id;
8          fail[i]=id;
9      }
10 }
11 /*以字串B匹配字串A，傳回匹配成功的數量(用B的fail)*/
12 int kmp_match(char *A,int lenA,char *B,int lenB,int *fail){
13     int id=-1,ans=0;
14     for(int i=0;i<lenA;++i){
15         while(~id&&B[id+1]!=A[i])id=fail[id];
16         if(B[id+1]==A[i])++id;
17         if(id==lenB-1){ /*匹配成功*/
18             ++ans, id=fail[id];
19         }
20     }
21     return ans;
22 }

```

7.4 manacher

```

1  //原字串: asdsasdsa
2  //先把字串變成這樣: @#a#s#d#s#a#s#d#s#a#
3  void manacher(char *s,int len,int *z){
4      int l=0,r=0;
5      for(int i=1;i<len;++i){
6          z[i]=r>i?min(z[2*i-1],r-i):1;
7          while(s[i+z[i]]==s[i-z[i]])++z[i];
8          if(z[i]+i>r)r=z[i]+i,l=i;
9      } /*ans = max(z)-1
10 }

```

7.5 minimal string rotation

```

1  int min_string_rotation(const string &s){
2      int n=s.size(),i=0,j=1,k=0;
3      while(i<n&&j<n&&k<n){
4          int t=s[(i+k)%n]-s[(j+k)%n];
5          ++k;
6          if(t){
7              if(t>0)i+=k;
8              else j+=k;
9              if(i==j)++j;
10             k=0;
11         }
12     }
13     return min(i,j); /*最小循環表示法起始位置
14 }

```

7.6 reverseBWT

```

1  const int MAXN = 305, MAXC = 'Z';
2  int ranks[MAXN], tots[MAXC], first[MAXC];
3  void rankBWT(const string &bw){
4      memset(ranks,0,sizeof(int)*bw.size());
5      memset(tots,0,sizeof(tots));
6      for(size_t i=0;i<bw.size();++i)
7          ranks[i] = tots[ bw[i] ]++;
8  }
9  void firstCol(){
10     memset(first,0,sizeof(first));
11     int totc = 0;
12     for(int c='A';c<='Z';++c){
13         if(!tots[c]) continue;
14         first[c] = totc;
15         totc += tots[c];
16     }
17 }
18 string reverseBwt(string bw,int begin){
19     rankBWT(bw, firstCol());
20     int i = begin; /*原字串最後一個元素的位置
21     string res;
22     do{
23         char c = bw[i];
24         res = c + res;
25         i = first[ bw[i] ] + ranks[i];
26     }while( i != begin );

```

```

27 | return res;
28 | }

```

7.7 suffix array lcp

```

1 | #define radix_sort(x,y){\
2 |   for(i=0;i<A;++i)c[i]=0;\
3 |   for(i=0;i<n;++i)c[x[y[i]]]++;\
4 |   for(i=1;i<A;++i)c[i]+=c[i-1];\
5 |   for(i=n-1;~i;--i)sa[--c[x[y[i]]]]=y[i];\
6 | }
7 | #define AC(r,a,b)\
8 |   r[a]!=(r[b]||a+k>n||r[a+k]!=(r[b+k]
9 | void suffix_array(const char *s,int n,int *
10 |   sa,int *rank,int *tmp,int *c){
11 |   int A='z'+1,i,k,id=0;
12 |   for(i=0;i<n;++i)rank[tmp[i]=i]=s[i];
13 |   radix_sort(rank,tmp);
14 |   for(k=1;id<n-1;k<=1){
15 |     for(id=0,i=n-k;i<n;++i)tmp[id++]=i;
16 |     for(i=0;i<n;++i)
17 |       if(sa[i]>=k)tmp[id++]=sa[i]-k;
18 |     radix_sort(rank,tmp);
19 |     swap(rank,tmp);
20 |     for(rank[sa[0]]=id=0,i=1;i<n;++i)
21 |       rank[sa[i]]=id+AC(tmp,sa[i-1],sa[i]);
22 |     A=id+1;
23 |   }
24 | //h: 高度數組 sa: 後綴數組 rank: 排名
25 | void suffix_array_lcp(const char *s,int len,
26 |   int *h,int *sa,int *rank){
27 |   for(int i=0;i<len;++i)rank[sa[i]]=i;
28 |   for(int i=0,k=0;i<len;++i){
29 |     if(rank[i]==0)continue;
30 |     if(k)--k;
31 |     while(s[i+k]==s[sa[rank[i]-1]+k])++k;
32 |     h[rank[i]]=k;
33 |   }
34 |   h[0]=0; // h[k]=Lcp(sa[k],sa[k-1]);

```

7.8 Z

```

1 | void z_alg(char *s,int len,int *z){
2 |   int l=0,r=0;
3 |   z[0]=len;
4 |   for(int i=1;i<len;++i){
5 |     z[i]=i>r?0:(i-1+z[i-1]<z[l]?z[i-1]:r-i
6 |       +1);
7 |     while(i+z[i]<len&&s[i+z[i]]==s[z[i]]++)z
8 |       [i];
9 |     if(i+z[i]-1>r)r=i+z[i]-1,l=i;

```

8 Tarjan

8.1 dominator tree

```

1 | struct dominator_tree{
2 |   static const int MAXN=5005;
3 |   int n; // 1-base
4 |   vector<int> G[MAXN], rG[MAXN];
5 |   int pa[MAXN], dfn[MAXN], id[MAXN], dfnCnt;
6 |   int semi[MAXN], idom[MAXN], best[MAXN];
7 |   vector<int> tree[MAXN]; // tree here
8 |   void init(int _n){
9 |     n = _n;
10 |    for(int i=1; i<=n; ++i)
11 |      G[i].clear(), rG[i].clear();
12 |  }
13 |  void add_edge(int u, int v){
14 |    G[u].push_back(v);
15 |    rG[v].push_back(u);
16 |  }
17 |  void dfs(int u){
18 |    id[dfn[u]]=++dfnCnt;
19 |    for(auto v:G[u]) if(!dfn[v])
20 |      dfs(v), pa[dfn[v]]=dfn[u];
21 |  }
22 |  int find(int y, int x){
23 |    if(y <= x) return y;
24 |    int tmp = find(pa[y], x);
25 |    if(semi[best[y]] > semi[best[pa[y]]])
26 |      best[y] = best[pa[y]];
27 |    return pa[y] = tmp;
28 |  }
29 |  void tarjan(int root){
30 |    dfnCnt = 0;
31 |    for(int i=1; i<=n; ++i){
32 |      dfn[i] = idom[i] = 0;
33 |      tree[i].clear();
34 |      best[i] = semi[i] = i;
35 |    }
36 |    dfs(root);
37 |    for(int i=dfnCnt; i>1; --i){
38 |      int u = id[i];
39 |      for(auto v:rG[u]) if(v=dfn[v]){
40 |        find(v,i);
41 |        semi[i]=min(semi[i], semi[best[v]]);
42 |      }
43 |      tree[semi[i]].push_back(i);
44 |      for(auto v:tree[pa[i]]){
45 |        find(v, pa[i]);
46 |        idom[v] = semi[best[v]]==pa[i]
47 |          ? pa[i] : best[v];
48 |      }
49 |      tree[pa[i]].clear();
50 |    }
51 |    for(int i=2; i<=dfnCnt; ++i){
52 |      if(idom[i] != semi[i])
53 |        idom[i] = idom[idom[i]];
54 |      tree[id[idom[i]]].push_back(i);
55 |    }
56 |  }
57 | }dom;

```

8.2 tnfsb017 2 sat

```

1 | #include<bits/stdc++.h>
2 | using namespace std;
3 | #define MAXN 8001
4 | #define MAXN2 MAXN*4
5 | #define n(X) ((X)+2*N)
6 | vector<int> v[MAXN2], rv[MAXN2], vis_t;
7 | int N,M;
8 | void addedge(int s,int e){
9 |   v[s].push_back(e);
10 |   rv[e].push_back(s);
11 | }
12 | int scc[MAXN2];
13 | bool vis[MAXN2]={false};
14 | void dfs(vector<int> *uv,int n,int k=-1){
15 |   vis[n]=true;
16 |   for(int i=0;i<uv[n].size();++i)
17 |     if(!vis[uv[n][i]])
18 |       dfs(uv,uv[n][i],k);
19 |   if(uv==v)vis_t.push_back(n);
20 |   scc[n]=k;
21 | }
22 | void solve(){
23 |   for(int i=1;i<=N;++i){
24 |     if(!vis[i])dfs(v,i);
25 |     if(!vis[n(i)])dfs(v,n(i));
26 |   }
27 |   memset(vis,0,sizeof(vis));
28 |   int c=0;
29 |   for(int i=vis_t.size()-1;i>=0;--i)
30 |     if(!vis[vis_t[i]])
31 |       dfs(rv,vis_t[i],c++);
32 | }
33 | int main(){
34 |   int a,b;
35 |   scanf("%d%d",&N,&M);
36 |   for(int i=1;i<=N;++i){
37 |     // (A or B)&(!A & !B) A^B
38 |     a=i*2-1;
39 |     b=i*2;
40 |     addedge(n(a),b);
41 |     addedge(n(b),a);
42 |     addedge(a,n(b));
43 |     addedge(b,n(a));
44 |   }
45 |   while(M--){
46 |     scanf("%d%d",&a,&b);
47 |     a = a>0?a*2-1:-a*2;
48 |     b = b>0?b*2-1:-b*2;
49 |     // A or B
50 |     addedge(n(a),b);
51 |     addedge(n(b),a);
52 |   }
53 |   solve();
54 |   bool check=true;
55 |   for(int i=1;i<=2*N;++i)
56 |     if(scc[i]==scc[n(i)])
57 |       check=false;
58 |   if(check){
59 |     printf("%d\n",N);
60 |     for(int i=1;i<=2*N;i+=2){
61 |       if(scc[i]>scc[i+2*N]) putchar('+');
62 |       else putchar('-');
63 |     }

```

```

64 |   puts("");
65 | }else puts("0");
66 | return 0;
67 | }

```

8.3 橋連通分量

```

1 | #define N 1005
2 | struct edge{
3 |   int u,v;
4 |   bool is_bridge;
5 |   edge(int u=0,int v=0):u(u),v(v),is_bridge
6 |     (0){}
7 | };
8 | vector<edge> E;
9 | vector<int> G[N]; // 1-base
10 | int low[N],vis[N],Time;
11 | int bcc_id[N],bridge_cnt,bcc_cnt; // 1-base
12 | int st[N],top; // BCC用
13 | void add_edge(int u,int v){
14 |   G[u].push_back(E.size());
15 |   E.emplace_back(u,v);
16 |   G[v].push_back(E.size());
17 |   E.emplace_back(v,u);
18 | }
19 | void dfs(int u,int re=-1){ // u當前點 · re為u連
20 |   接前一個點的邊
21 |   int v;
22 |   low[u]=vis[u]=++Time;
23 |   st[top++]=u;
24 |   for(int e:G[u]){
25 |     v=E[e].v;
26 |     if(!vis[v]){
27 |       dfs(v,e^1); // e^1 反向邊
28 |       low[u]=min(low[u],low[v]);
29 |       if(vis[u]<low[v]){
30 |         E[e].is_bridge=E[e^1].is_bridge=1;
31 |         ++bridge_cnt;
32 |       }
33 |     }else if(vis[v]<vis[u]&&e!=re)
34 |       low[u]=min(low[u],vis[v]);
35 |   }
36 |   if(vis[u]==low[u]){ // 處理 BCC
37 |     ++bcc_cnt; // 1-base
38 |     do bcc_id[v=st[--top]]=bcc_cnt; // 每個點
39 |       所在的 BCC
40 |     while(v!=u);
41 |   }
42 | }
43 | void bcc_init(int n){
44 |   Time=bcc_cnt=bridge_cnt=top=0;
45 |   E.clear();
46 |   for(int i=1;i<=n;++i){
47 |     G[i].clear();
48 |     vis[i]=bcc_id[i]=0;
49 |   }

```

8.4 雙連通分量 & 割點

```

1 #define N 1005
2 vector<int> G[N]; // 1-base
3 vector<int> bcc[N]; // 存每塊雙連通分量的點
4 int low[N], vis[N], Time;
5 int bcc_id[N], bcc_cnt; // 1-base
6 bool is_cut[N]; // 是否為割點
7 int st[N], top;
8 void dfs(int u, int pa=-1) { // u當前點, pa父親
9     int t, child=0;
10    low[u]=vis[u]=++Time;
11    st[top++]=u;
12    for(int v:G[u]){
13        if(!vis[v]){
14            dfs(v,u), ++child;
15            low[u]=min(low[u], low[v]);
16            if(vis[u]<=low[v]){
17                is_cut[u]=1;
18                bcc[++bcc_cnt].clear();
19                do{
20                    bcc_id[t=st[--top]]=bcc_cnt;
21                    bcc[bcc_cnt].push_back(t);
22                }while(t!=v);
23                bcc_id[u]=bcc_cnt;
24                bcc[bcc_cnt].push_back(u);
25            }
26        } else if(vis[v]<vis[u]&&v!=pa) // 反向邊
27            low[u]=min(low[u], vis[v]);
28    } // u是dfs樹的根要特判
29    if(pa==-1&&child<2) is_cut[u]=0;
30 }
31 void bcc_init(int n){
32     Time=bcc_cnt=top=0;
33     for(int i=1; i<=n; ++i){
34         G[i].clear();
35         is_cut[i]=vis[i]=bcc_id[i]=0;
36     }
37 }

```

9 Tree Problem

9.1 HeavyLight

```

1 #include<vector>
2 #define MAXN 100005
3 int siz[MAXN], max_son[MAXN], pa[MAXN], dep[
4     MAXN];
5 int link_top[MAXN], link[MAXN], cnt;
6 vector<int> G[MAXN];
7 void find_max_son(int u){
8     siz[u]=1;
9     max_son[u]=-1;
10    for(auto v:G[u]){
11        if(v==pa[u]) continue;
12        pa[v]=u;
13        dep[v]=dep[u]+1;
14        find_max_son(v);

```

```

15        if(max_son[u]==-1 || siz[v]>siz[max_son[u]
16            ]) max_son[u]=v;
17        siz[u]+=siz[v];
18    }
19    void build_link(int u, int top){
20        link[u]=++cnt;
21        link_top[u]=top;
22        if(max_son[u]==-1) return;
23        build_link(max_son[u], top);
24        for(auto v:G[u]){
25            if(v==max_son[u] || v==pa[u]) continue;
26            build_link(v, v);
27        }
28    }
29    int find_lca(int a, int b){
30        // 求LCA, 可以在過程中對區間進行處理
31        int ta=link_top[a], tb=link_top[b];
32        while(ta!=tb){
33            if(dep[ta]<dep[tb]){
34                swap(ta, tb);
35                swap(a, b);
36            }
37            // 這裡可以對a所在的鏈做區間處理
38            // 區間為(Link[ta], Link[a])
39            ta=link_top[a=pa[ta]];
40        } // 最後a,b會在同一條鏈, 若a!=b還要在進行一
41            次區間處理
42        return dep[a]<dep[b]?a:b;

```

9.2 LCA

```

1 const int MAXN=100000; // 1-base
2 const int MLG=17; // Log2(MAXN)+1;
3 int pa[MLG+2][MAXN+5];
4 int dep[MAXN+5];
5 vector<int> G[MAXN+5];
6 void dfs(int x, int p=0) { // dfs(root);
7     pa[0][x]=p;
8     for(int i=0; i<=MLG; ++i)
9         pa[i+1][x]=pa[i][pa[i][x]];
10    for(auto &i:G[x]){
11        if(i==p) continue;
12        dep[i]=dep[x]+1;
13        dfs(i, x);
14    }
15 }
16 inline int jump(int x, int d){
17     for(int i=0; i<=MLG; ++i)
18         if((d>>i)&1) x=pa[i][x];
19     return x;
20 }
21 inline int find_lca(int a, int b){
22     if(dep[a]>dep[b]) swap(a, b);
23     b=jump(b, dep[b]-dep[a]);
24     if(a==b) return a;
25     for(int i=MLG; i>=0; --i){
26         if(pa[i][a]!=pa[i][b]){
27             a=pa[i][a];
28             b=pa[i][b];

```

```

29     }
30     }
31     return pa[0][a];
32 }

```

9.3 link cut tree

```

1 struct splay_tree{
2     int ch[2], pa; // 子節點跟父母
3     bool rev; // 反轉的懶惰標記
4     splay_tree(): pa(0), rev(0) { ch[0]=ch[1]=0; }
5 };
6 vector<splay_tree> nd;
7 // 有的時候用vector會TLE, 要注意
8 // 這邊以node[0]作為null節點
9 bool isroot(int x) { // 判斷是否為這棵splay
10    tree的根
11    return nd[nd[x].pa].ch[0]!=x&&nd[nd[x].pa
12        ].ch[1]!=x;
13 }
14 void down(int x) { // 懶惰標記下推
15     if(nd[x].rev){
16         if(nd[x].ch[0]nd[nd[x].ch[0]].rev^=1;
17         if(nd[x].ch[1]nd[nd[x].ch[1]].rev^=1;
18         swap(nd[x].ch[0], nd[x].ch[1]);
19         nd[x].rev=0;
20     }
21 }
22 void push_down(int x) { // 所有祖先懶惰標記下推
23     if(!isroot(x)) push_down(nd[x].pa);
24     down(x);
25 }
26 void up(int x) { // 將子節點的資訊向上更新
27 }
28 void rotate(int x) { // 旋轉, 會自行判斷轉的方
29    向
30    int y=nd[x].pa, z=nd[y].pa, d=(nd[y].ch[1]==
31        x);
32    nd[x].pa=z;
33    if(!isroot(y)) nd[z].ch[nd[z].ch[1]==y]=x;
34    nd[y].ch[d]=nd[x].ch[d^1];
35    nd[nd[y].ch[d]].pa=y;
36    nd[y].pa=x, nd[x].ch[d^1]=y;
37    up(y), up(x);
38 }
39 void splay(int x) { // 將x伸展到splay tree的根
40    push_down(x);
41    while(!isroot(x)){
42        int y=nd[x].pa;
43        if(!isroot(y)){
44            int z=nd[y].pa;
45            if((nd[z].ch[0]==y)^(nd[y].ch[0]==x))
46                rotate(y);
47            else rotate(x);
48        }
49        rotate(x);
50    }
51 }
52 int access(int x){
53     int last=0;
54     while(x){
55         splay(x);

```

```

56         nd[x].ch[1]=last;
57         up(x);
58         last=x;
59         x=nd[x].pa;
60     }
61     return last; // access後splay tree的根
62 }
63 void access(int x, bool is=0) { // is=0就是一般
64    的access
65    int last=0;
66    while(x){
67        splay(x);
68        if(is&&!nd[x].pa){
69            // printf("%d\n", max(nd[last].ma, nd[nd[
70                x].ch[1]].ma));
71        }
72        nd[x].ch[1]=last;
73        up(x);
74        last=x;
75        x=nd[x].pa;
76    }
77 }
78 void query_edge(int u, int v){
79     access(u);
80     access(v, 1);
81 }
82 void make_root(int x){
83     access(x), splay(x);
84     nd[x].rev^=1;
85 }
86 void make_root(int x){
87     nd[access(x)].rev^=1;
88     splay(x);
89 }
90 void cut(int x, int y){
91     make_root(x);
92     access(y);
93     splay(y);
94     nd[y].ch[0]=0;
95     nd[x].pa=0;
96 }
97 void cut_parents(int x){
98     access(x);
99     splay(x);
100    nd[nd[x].ch[0]].pa=0;
101    nd[x].ch[0]=0;
102 }
103 void link(int x, int y){
104     make_root(x);
105     nd[x].pa=y;
106 }
107 int find_root(int x){
108     x=access(x);
109     while(nd[x].ch[0]) x=nd[x].ch[0];
110     splay(x);
111     return x;
112 }
113 int query(int u, int v){
114     // 傳回uv路徑splay tree的根結點
115     // 這種寫法無法求LCA
116     make_root(u);
117     return access(v);
118 }
119 int query_lca(int u, int v){

```

```

112 //假設求鏈上點權的總和，sum是子樹的權重和，
    data是節點的權重
113 access(u);
114 int lca=access(v);
115 splay(u);
116 if(u==lca){
117     //return nd[lca].data+nd[nd[lca].ch[1]].
    sum
118 }else{
119     //return nd[lca].data+nd[nd[lca].ch[1]].
    sum+nd[u].sum
120 }
121 }
122 struct EDGE{
123     int a,b,w;
124 }e[10005];
125 int n;
126 vector<pair<int,int>> G[10005];
127 //first表示子節點，second表示邊的編號
128 int pa[10005],edge_node[10005];
129 //pa是父母節點，暫存用的，edge_node是每個編
    被存在哪個點裡面的陣列
130 void bfs(int root){
131     //在建構的時候把每個點都設成一個splay tree
132     queue<int> q;
133     for(int i=1;i<=n;++i)pa[i]=0;
134     q.push(root);
135     while(q.size()){
136         int u=q.front();
137         q.pop();
138         for(auto P:G[u]){
139             int v=P.first;
140             if(v!=pa[u]){
141                 pa[v]=u;
142                 nd[v].pa=u;
143                 nd[v].data=e[P.second].w;
144                 edge_node[P.second]=v;
145                 up(v);
146                 q.push(v);
147             }
148         }
149     }
150 }
151 void change(int x,int b){
152     splay(x);
153     //nd[x].data=b;
154     up(x);
155 }

```

9.4 POJ tree

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define MAXN 10005
4 int n,k;
5 vector<pair<int,int>> g[MAXN];
6 int size[MAXN];
7 bool vis[MAXN];
8 inline void init(){
9     for(int i=0;i<=n;++i){
10         g[i].clear();
11         vis[i]=0;

```

```

12     }
13 }
14 void get_dis(vector<int> &dis,int u,int pa,
    int d){
15     dis.push_back(d);
16     for(size_t i=0;i<g[u].size();++i){
17         int v=g[u][i].first,w=g[u][i].second;
18         if(v!=pa&&!vis[v])get_dis(dis,v,u,d+w);
19     }
20 }
21 vector<int> dis;//這東西如果放在函數裡會TLE
22 int cal(int u,int d){
23     dis.clear();
24     get_dis(dis,u,-1,d);
25     sort(dis.begin(),dis.end());
26     int l=0,r=dis.size()-1,res=0;
27     while(l<r){
28         while(l<r&&dis[l]+dis[r]>k)--r;
29         res+=r-(l++);
30     }
31     return res;
32 }
33 pair<int,int> tree_centroid(int u,int pa,
    const int sz){
34     size[u]=1;//找樹重心，second是重心
35     pair<int,int> res(INT_MAX,-1);
36     int ma=0;
37     for(size_t i=0;i<g[u].size();++i){
38         int v=g[u][i].first;
39         if(v==pa||vis[v])continue;
40         res=min(res,tree_centroid(v,u,sz));
41         size[u]+=size[v];
42         ma=max(ma,size[v]);
43     }
44     ma=max(ma,sz-size[u]);
45     return min(res,make_pair(ma,u));
46 }
47 int tree_DC(int u,int sz){
48     int center=tree_centroid(u,-1,sz).second;
49     int ans=cal(center,0);
50     vis[center]=1;
51     for(size_t i=0;i<g[center].size();++i){
52         int v=g[center][i].first,w=g[center][i].
            second;
53         if(vis[v])continue;
54         ans-=cal(v,w);
55         ans+=tree_DC(v,size[v]);
56     }
57     return ans;
58 }
59 int main(){
60     while(scanf("%d%d",&n,&k),n||k){
61         init();
62         for(int i=1;i<=n;++i){
63             int u,v,w;
64             scanf("%d%d%d",&u,&v,&w);
65             g[u].push_back(make_pair(v,w));
66             g[v].push_back(make_pair(u,w));
67         }
68         printf("%d\n",tree_DC(1,n));
69     }
70     return 0;
71 }

```

10 default

10.1 debug

```

1 #ifndef DEBUG
2 #define dbg(...) {\
3     fprintf(stderr,"%s - %d : (%s) = ",
        __PRETTY_FUNCTION__,__LINE__,#
        __VA_ARGS__);\
4     _DO(__VA_ARGS__);\
5 }
6 template<typename I> void _DO(I&&x){cerr<<x
    <<endl;}
7 template<typename I,typename...T> void _DO(I
    &&x,T&&...tail){cerr<<x<<" ";_DO(tail
    ...);}
8 #else
9 #define dbg(...)
10 #endif

```

10.2 ext

```

1 #include<bits/extc++.h>
2 #include<ext/pd_ds/assoc_container.hpp>
3 #include<ext/pd_ds/tree_policy.hpp>
4 using namespace __gnu_cxx;
5 using namespace __gnu_pbds;
6 template<typename T>
7 using pbds_set = tree<T,null_type,less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update>;
8 template<typename T,typename U>
9 using pbds_map = tree<T,U,less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update>;
10 using heap=__gnu_pbds::priority_queue<int>;
11 //s.find_by_order(1);//0 base
12 //s.order_of_key(1);

```

10.3 IncStack

```

1 //Magic
2 #pragma GCC optimize "Ofast"
3 //stack resize,change esp to rsp if 64-bit
    system
4 asm("mov %0,%esp\n"::"g"(mem+1000000));
5 -Wl,--stack,214748364 -trigraphs
6 #pragma comment(linker,"/STACK
    :1024000000,1024000000")
7 //linux stack resize
8 #include<sys/resource.h>
9 void increase_stack(){
10     const rlim_t ks=64*1024*1024;
11     struct rlimit rl;
12     int res=getrlimit(RLIMIT_STACK,&rl);
13     if(!res&&rl.rlim_cur<ks){
14         rl.rlim_cur=ks;

```

```

15     res=setrlimit(RLIMIT_STACK,&rl);
16 }
17 }

```

10.4 input

```

1 inline int read(){
2     int x=0; bool f=0; char c=getchar();
3     while(ch<'0'||'9'<ch)f|=ch=='-',ch=getchar
        ();
4     while('0'<=ch&&ch<='9')x=x*10-'0'+ch,ch=
        getchar();
5     return f?-x:x;
6 }
7 //#!/bin/bash
8 //g++ -std=c++11 -O2 -Wall -Wextra -Wno-
    unused-result -DDEBUG $1 && ./a.out
9 // -fsanitize=address -fsanitize=undefined
    -fsanitize=return

```

11 other

11.1 WhatDay

```

1 int whatday(int y,int m,int d){
2     if(m<2)m+=12,--y;
3     if(y<1752||y==1752&&m<9||y==1752&&m==9&&d
        <3)
4         return (d+2*m+3*(m+1)/5+y+y/4+5)%7;
5     return (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400
        %7);
6 }

```

11.2 上下最大正方形

```

1 void solve(int n,int a[],int b[]){// 1-base
2     int ans=0;
3     deque<int> da,db;
4     for(int l=1,r=1;r<=n;++r){
5         while(da.size()&&a[da.back()]>=a[r]){
6             da.pop_back();
7         }
8         da.push_back(r);
9         while(db.size()&&b[db.back()]>=b[r]){
10             db.pop_back();
11         }
12         db.push_back(r);
13         for(int d=a[da.front()]+b[db.front()];r-
            l+1>d;++l){
14             if(da.front()==1)da.pop_front();
15             if(db.front()==1)db.pop_front();
16             if(da.size()&&db.size()){
17                 d=a[da.front()]+b[db.front()];
18             }
19         }

```

```

20 |     ans=max(ans,r-1+1);
21 | }
22 | printf("%d\n",ans);
23 | }

```

11.3 最大矩形

```

1 | LL max_rectangle(vector<int> s){
2 |     stack<pair<int,int> > st;
3 |     st.push(make_pair(-1,0));
4 |     s.push_back(0);
5 |     LL ans=0;
6 |     for(size_t i=0;i<s.size();++i){
7 |         int h=s[i];
8 |         pair<int,int> now=make_pair(h,i);
9 |         while(h<st.top().first){
10 |             now=st.top();
11 |             st.pop();
12 |             ans=max(ans,(LL)(i-now.second)*now.first);
13 |         }
14 |         if(h>st.top().first){
15 |             st.push(make_pair(h,now.second));
16 |         }
17 |     }
18 |     return ans;
19 | }

```

12 other language

12.1 java

12.1.1 文件操作

```

1 | import java.io.*;
2 | import java.util.*;
3 | import java.math.*;
4 | import java.text.*;
5 |
6 | public class Main{
7 |
8 |     public static void main(String args[]){
9 |         throws FileNotFoundException,
10 |         IOException
11 |         Scanner sc = new Scanner(new FileReader(
12 |             "a.in"));
13 |         PrintWriter pw = new PrintWriter(new
14 |             FileWriter("a.out"));
15 |         int n,m;
16 |         n=sc.nextInt();//读入下一个INT
17 |         m=sc.nextInt();
18 |
19 |         for(ci=1; ci<=c; ++ci){
20 |             pw.println("Case #"+ci+": easy for
21 |                 output");
22 |         }
23 |     }
24 | }

```

```

19 |     pw.close();//关闭流并释放，这个很重要，
20 |     // 否则是没有输出的
21 |     sc.close();//关闭流并释放
22 | }

```

12.1.2 优先队列

```

1 | PriorityQueue queue = new PriorityQueue( 1,
2 |     new Comparator(){
3 |         public int compare( Point a, Point b ){
4 |             if( a.x < b.x || a.x == b.x && a.y < b.y )
5 |                 return -1;
6 |             else if( a.x == b.x && a.y == b.y )
7 |                 return 0;
8 |             else return 1;
9 |         }
10 |     });

```

12.1.3 Map

```

1 | Map map = new HashMap();
2 | map.put("sa","dd");
3 | String str = map.get("sa").toString();
4 |
5 | for(Object obj : map.keySet()){
6 |     Object value = map.get(obj);
7 | }

```

12.1.4 sort

```

1 | static class cmp implements Comparator{
2 |     public int compare(Object o1,Object o2){
3 |         BigInteger b1=(BigInteger)o1;
4 |         BigInteger b2=(BigInteger)o2;
5 |         return b1.compareTo(b2);
6 |     }
7 | }
8 | public static void main(String[] args)
9 |     throws IOException{
10 |     Scanner cin = new Scanner(System.in);
11 |     int n;
12 |     n=cin.nextInt();
13 |     BigInteger[] seg = new BigInteger[n];
14 |     for (int i=0;i<n;i++){
15 |         seg[i]=cin.nextBigInteger();
16 |         Arrays.sort(seg,new cmp());
17 |     }

```

12.2 python heap

```

1 | import heapq
2 |
3 | heap = [7,1,2,2]
4 | heapq.heapify(heap)
5 | print(heap) # [1, 2, 2, 7]
6 | heapq.heappush(heap, 5)
7 | print(heap) # [1, 2, 2, 7, 5]
8 | print(heapq.heappop(heap)) # 1
9 | print(heap) # [2, 2, 5, 7]

```

12.3 python input

```

1 | ans = sum(map(float, input().split()))
2 | # input: 1.1 2.2 3.3 4.4 5.5
3 | print(ans) # 16.5
4 |
5 | (n, m) = map(int, input().split()) # 300 200
6 | print(n * m) # 60000
7 |
8 | Arr = list(map(int, input().split()))
9 | # input: 1 2 3 4 5
10 | print(Arr) # [1, 2, 3, 4, 5]

```

12.4 python output

```

1 | hello = 'Hello'
2 | world = 7122
3 | print(f'{hello} {world}') # Hello 7122
4 |
5 | import math
6 | print(f'PI is approximately {math.pi:.3f}.')
7 | # PI is approximately 3.142.
8 |
9 | print('AAA {} BBB {}'.format('Jin', 'KeLa'))
10 | # AAA Jin BBB "KeLa!"
11 |
12 | hello = 'hello, world\n'
13 | hellos = repr(hello)
14 | print(hellos) # 'hello, world\n'
15 |
16 | x = 32.5
17 | y = 40000
18 | print(repr((x, y, ('spam', 'eggs'))))
19 | # "(32.5, 40000, ('spam', 'eggs'))"
20 |
21 | x = 7
22 | print(eval('3 * x')) # 21

```

13 zformula

13.1 formula

13.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

13.1.2 圖論

- 對於平面圖， $F = E - V + C + 1$ ， C 是連通分量數
- 對於平面圖， $E < 3V - 6$
- 對於連通圖 G ，最大獨立點集的大小設為 $I(G)$ ，最大匹配大小設為 $M(G)$ ，最小點覆蓋設為 $C_v(G)$ ，最小邊覆蓋設為 $C_e(G)$ ，對於任意連通圖：

- $I(G) + C_v(G) = |V|$
- $M(G) + C_e(G) = |V|$

- 對於連通二分圖：

- $I(G) = C_v(G)$
- $M(G) = C_e(G)$

- 最大權閉合圖：

- $C(u, v) = \infty, (u, v) \in E$
- $C(S, v) = W_v, W_v > 0$
- $C(v, T) = -W_v, W_v < 0$
- $ans = \sum_{W_v > 0} W_v - flow(S, T)$

- 最大密度子圖：

- 求 $max \left(\frac{W_e + W_v}{|V|} \right), e \in E', v \in V'$
- $U = \sum_{v \in V} 2W_v + \sum_{e \in E} W_e$
- $C(u, v) = W_{(u, v)}, (u, v) \in E$ ，雙向邊
- $C(S, v) = U, v \in V$
- $D_u = \sum_{(u, v) \in E} W_{(u, v)}$
- $C(v, T) = U + 2g - D_v - 2W_v, v \in V$
- 二分搜 g ：
 $l = 0, r = U, eps = 1/n^2$
 if $((U \times |V| - flow(S, T))/2 > 0) l = mid$
 else $r = mid$
- $ans = min_cut(S, T)$
- $|E| = 0$ 要特殊判斷

- 弦圖：

- 點數大於 3 的環都要有一條弦
- 完美消除序列從後往前依次給每個點染色，給每個點染上可以染的最小顏色
- 最大團大小 = 色數
- 最大獨立集：完美消除序列從前往後能選就選
- 最小團覆蓋：最大獨立集的點和他延伸的邊構成
- 區間圖是弦圖
- 區間圖的完美消除序列：將區間按造又端點由小到大排序
- 區間圖染色：用線段樹做

13.1.3 dinic 特殊圖複雜度

- 單位流： $O \left(\min \left(V^{3/2}, E^{1/2} \right) E \right)$
- 二分圖： $O \left(V^{1/2} E \right)$

13.1.4 0-1 分數規劃

$x_i \in \{0, 1\}$ · x_i 可能會有其他限制 · 求 $\max \left(\frac{\sum B_i x_i}{\sum C_i x_i} \right)$

1. $D(i, g) = B_i - g \times C_i$
2. $f(g) = \sum D(i, g) x_i$
3. $f(g) = 0$ 時 g 為最佳解 · $f(g) < 0$ 沒有意義
4. 因為 $f(g)$ 單調可以二分搜 g
5. 或用 Dinkelbach 通常比較快

```

1 binary_search(){
2   while(r-l>eps){
3     g=(l+r)/2;
4     for(i:所有元素)D[i]=B[i]-g*C[i];//D(i,g)
5     找出一組合法x[i]使f(g)最大;
6     if(f(g)>0) l=g;
7     else r=g;
8   }
9   Ans = r;
10 }
11 Dinkelbach(){
12   g=任意狀態 (通常設為0);
13   do{
14     Ans=g;
15     for(i:所有元素)D[i]=B[i]-g*C[i];//D(i,g)
16     找出一組合法x[i]使f(g)最大;
17     p=0,q=0;
18     for(i:所有元素)
19       if(x[i])p+=B[i],q+=C[i];
20     g=p/q;//更新解 · 注意q=0的情況
21   }while(abs(Ans-g)>EPS);
22   return Ans;
23 }
```

13.1.5 學長公式

1. $\sum_{d|n} \phi(n) = n$
2. $g(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) \times g(n/d)$
3. Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
4. $\gamma = 0.57721566490153286060651209008240243104215$
5. 格雷碼 $= n \oplus (n >> 1)$
6. $SG(A+B) = SG(A) \oplus SG(B)$
7. 選轉矩陣 $M(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

13.1.6 基本數論

1. $\sum_{d|n} \mu(n) = [n == 1]$
2. $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
3. $\sum_{i=1}^n \sum_{j=1}^m \text{互質數量} = \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
4. $\sum_{i=1}^n \sum_{j=1}^n \text{lcm}(i, j) = n \sum_{d|n} d \times \phi(d)$

13.1.7 排組公式

1. k 卡特蘭 $\frac{C^{kn}}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
2. $H(n, m) \cong x_1 + x_2 \dots + x_n = k, num = C_k^{n+k-1}$
3. Stirling number of $2^{nd}, n$ 人分 k 組方法數目
 - (a) $S(0, 0) = S(n, n) = 1$
 - (b) $S(n, 0) = 0$
 - (c) $S(n, k) = kS(n-1, k) + S(n-1, k-1)$
4. Bell number, n 人分任意多組方法數目

- (a) $B_0 = 1$
- (b) $B_n = \sum_{i=0}^n S(n, i)$
- (c) $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
- (d) $B_{p+n} \equiv B_n + B_{n+1} \pmod{p}$, p is prime
- (e) $B_p^{m+n} \equiv mB_n + B_{n+1} \pmod{p}$, p is prime
- (f) From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$

5. Derangement, 錯排, 沒有人在自己位置上

- (a) $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \dots + (-1)^n \frac{1}{n!})$
- (b) $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
- (c) From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$

6. Binomial Equality

- (a) $\sum_k \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{m+n}$
- (b) $\sum_k \binom{r}{m+k} \binom{s}{n+k} = \binom{l+s}{l-m+n}$
- (c) $\sum_k \binom{l}{m+k} \binom{s+k}{n} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$
- (d) $\sum_{k \leq l} \binom{l-k}{m} \binom{s}{n-k} (-1)^k = (-1)^{l+m} \binom{s-m-1}{l-n-m}$
- (e) $\sum_{0 \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
- (f) $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$
- (g) $\binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k}$
- (h) $\sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{n+1}$
- (i) $\sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$
- (j) $\sum_{k \leq m} \binom{m+r}{k} x^k y^k = \sum_{k \leq m} \binom{-r}{k} (-x)^k (x+y)^{m-k}$

13.1.8 冪次, 冪次和

1. $a^{b \% P} = a^{b \% \varphi(P) + \varphi(P)}, b \geq \varphi(P)$
2. $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
3. $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
4. $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
5. $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n+1$
6. $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
7. $\sum_{j=0}^m C_j^{m+1} B_j = 0, B_0 = 1$
8. 除了 $B_1 = -1/2$ · 剩下的奇數項都是 0
9. $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

13.1.9 Burnside's lemma

1. $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$
2. $X^g = t^{c(g)}$
3. G 表示有幾種轉法 · X^g 表示在那種轉法下 · 有幾種是會保持對稱的 · t 是顏色數 · $c(g)$ 是循環節不動的面數 ·
4. 正立方體塗三顏色 · 轉 0 有 3^6 個元素不變 · 轉 90 有 6 種 · 每種有 3^3 不變 · 180 有 $3 \times 3^4 \cdot 120(\text{角})$ 有 $8 \times 3^2 \cdot 180(\text{邊})$ 有 6×3^3 · 全部 $\frac{1}{24} (3^6 + 6 \times 3^3 + 3 \times 3^4 + 8 \times 3^2 + 6 \times 3^3) = \frac{57}{57}$

13.1.10 Count on a tree

1. Rooted tree: $s_{n+1} = \frac{1}{n} \sum_{i=1}^n (i \times a_i \times \sum_{j=1}^{\lfloor n/i \rfloor} a_{n+1-i \times j})$
2. Unrooted tree:
 - (a) Odd: $a_n - \sum_{i=1}^{n/2} a_i a_{n-i}$
 - (b) Even: $Odd + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$
3. Spanning Tree
 - (a) 完全圖 $n^n - 2$
 - (b) 一般圖 (Kirchhoff's theorem) $M[i][i] = \text{degree}(V_i), M[i][j] = -1, \text{if have } E(i, j), 0 \text{ if no edge. delete any one row and col in } A, ans = \det(A)$

14 Интернационал

14.1 ganadoQuote

```

1 ¡Allí está!
2 ¡Un forastero!
3 ¡Agarrenlo!
4 ¡Os voy a romper a pedazos!
5 ¡Cógelo!
6 ¡Te voy a hacer picadillo!
7 ¡Te voy a matar!
8 ¡Míralo, está herido!
9 ¡Sos cerdo!
10 ¿Dónde estás?
11 ¡Detrás de tí, imbécil!
12 ¡No dejes que se escape!
13 ¡Basta, hijo de puta!
14 Lord Saddler...
15
16 ¡Mátalo!
17 ¡Allí está!
18 Morir es vivir.
19 ¡Síííí, ¡Quiero matar!
20 Muere, muere, muere....
21 Cerebros, cerebros, cerebros...
22 Cógedlo, cógedlo, cógedlo...
23 Lord Saddler...
24 Dieciséis.
25
```

```

26 ¡Va por él!
27 ¡Muérete!
28 ¡Cógelo!
29 ¡Te voy a matar!
30 ¡Bloqueale el paso!
31 ¡Te cogí!
32 ¡No dejes que se escape!
33
34 ¿Qué carajo estás haciendo aquí? ¡Lárgate, cabrón!
35 Hay un rumor de que hay un extranjero entre nosotros.
36 Nuestro jefe se encargará de la rata.
37 Su "Las Plagas" es mucho mejor que la nuestra.
38 Tienes razón, es un hombre.
39 Usa los músculos.
40 Se vuelve loco!
41 ¡Hey, acá!
42 ¡Por aquí!
43 ¡El Gigante!
44 ¡Del Lago!
45 ¡Cógelo!
46 ¡Cógenlo!
47 ¡Allí!
48 ¡Rápido!
49 ¡Empieza a rezar!
50 ¡Mátenlos!
51 ¡Te voy a romper en pedazos!
52 ¡La campana!
53 Ya es hora de rezar.
54 Tenemos que irnos.
55 ¡Maldita sea, mierda!
56 ¡Ya es hora de aplastar!
57 ¡Mierda!
58 ¡Puedes correr, pero no te puedes esconder!
59 ¡Sos cerdo!
60 ¡Está en la trampa!
61 ¡Ah, que madre!
62 ¡Vámonos!
63 ¡Ándale!
64 ¡Cabrón!
65 ¡Coño!
66 ¡Agárrenlo!
67 Cógerlo, Cógerlo...
68 ¡Allí está, mátalo!
69 ¡No dejes que se escape de la isla vivo!
70 ¡Hasta luego!
71 ¡Rápido, es un intruso!
```

14.2 Интернационал

```
1 /*****
2 L'Internationale,
3 Sera le genre humain.
4
5
6
7
8
9
10
11
12
13
14
15
16 *****/
17 Вставай, проклятем заклеимный,
18 Весь мир голодных и рабов!
19 Кипит наш разум возмущенный
20 И в смертный бой вести готов.
21 Весь мир насилия мы разрушим
22 До основания, а затем
23 Мы наш, мы новый мир построим, –
24 Кто был ничем, тот станет всем.
25
26 Chorus
27 Это есть наш последний
28 И решительный бой;
29 С Интернационалом
30 Воспрянет род людской!
31
32 Никто не даст нам избавленья:
33 Ни бог, ни царь и не герой!
34 Добьёмся мы освобожденья
35 Своею собственной рукой.
36 Чтоб свергнуть гнёт рукой умелой,
37 Отвоевать своё добро, –
38 Вдуйте горн и куйте смело,
39 Пока железо горячо!
40
41 Chorus
42
43 Довольно кровь сосать, вампиры,
44 Тюрьмой, налогом, нищетой!
45 У вас — вся власть, все блага мира,
46 А наше право — звук пустой !
47 Мы жизнь построим по-иному –
48 И вот наш лозунг боевой:
49 Вся власть народу трудовому!
50 А дармоедов всех долой!
51
52 Chorus
53
54 Презренны вы в своём богатстве,
55 Угля и стали короли!
56 Вы ваши троны, туеядцы,
57 На наших спинах возвели.
58 Заводы, фабрики, палаты –
59 Всё нашим создано трудом.
60 Пора! Мы требуем возврата
61 Того, что взято грабёжом.
62
63 Chorus
```

```
64
65 Довольно королям в угоду
66 Дурманить нас в чаду войны!
67 Война тиранам! Мир Народу!
68 Бастуйте, армии сыны!
69 Когда ж тираны нас заставят
70 В бою геройски пасть за них –
71 Убийцы, в вас тогда направим
72 Мы жерла пушек боевых!
73
74 Chorus
75
76 Лишь мы, работники всемирной
77 Великой армии труда,
78 Владеть землёй имеем право,
79 Но паразиты — никогда!
80 И если гром великий грянет
81 Над сворой псов и палачей, –
82 Для нас всё так же солнце станет
83 Сиять огнём своих лучей.
84
85 Chorus
```

14.3 保佑

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15 //
16 //
17 //
18 //
19 //
20 //
21 //
22 //
23 //
24 //
25 //
26 //
27 //
28 //
29 //
30 //
31 //
32 //
33 //
34 //
35 //
36
37
38
```

```
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54 #
55 #
56 #
57 #
58 #
59 #
60 #
61 #
62 #
63 #
64 #
65 #
66 #
67 #
68 #
69 #
70 #
71 #
72 #
73 #
74 #
75
76
77 // ##
78 // ##
79 // ##
80 // ##
81 // ##
82 // ##
83 // ##
84 // ##
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //
93 //
94 //
95 //
96
97 //
98 //
99 //
```

```
100 //
101 //
102 //
103 //
104 //
105 //
106 //
107 //
108 //
109 //
110 //
```

```
100 //
101 //
102 //
103 //
104 //
105 //
106 //
107 //
108 //
109 //
110 //
```

```
100 //
101 //
102 //
103 //
104 //
105 //
106 //
107 //
108 //
109 //
110 //
```

ACM ICPC Team Reference - CCU- oshiete536senpai

Contents

1 Data Structure	1	3.8 MaximumClique	4	6.15 質因數分解	9	11 other	13
1.1 segment tree lazytag	1	3.9 MinimumMeanCycle	4	7 String	10	11.1 WhatDay	13
2 Flow	1	3.10 Rectilinear MST	4	7.1 AC 自動機	10	11.2 上下最大正方形	13
2.1 dinic	1	3.11 treeISO	4	7.2 hash	10	11.3 最大矩形	14
2.2 Gomory Hu	1	3.12 一般圖最小權完美匹配	4	7.3 KMP	10	12 other language	14
2.3 ISAP with cut	1	3.13 全局最小割	5	7.4 manacher	10	12.1 java	14
2.4 MinCostMaxFlow	1	3.14 弦圖完美消除序列	5	7.5 minimal string rotation	10	12.1.1 文件操作	14
3 Graph	2	3.15 最小斯坦納樹 DP	5	7.6 reverseBWT	10	12.1.2 優先隊列	14
3.1 Augmenting Path	2	3.16 最小樹形圖朱劉	5	7.7 suffix array lcp	11	12.1.3 Map	14
3.2 Augmenting Path multiple	2	3.17 穩定婚姻模板	6	7.8 Z	11	12.1.4 sort	14
3.3 blossom matching	2	4 Language	6	8 Tarjan	11	12.2 python heap	14
3.4 BronKerbosch	2	4.1 CNF	6	8.1 dominator tree	11	12.3 python input	14
3.5 graphISO	3	5 Linear Programming	6	8.2 tnfsb017 2 sat	11	12.4 python output	14
3.6 is planar	3	5.1 simplex	6	8.3 橋連通分量	11	13 zformula	14
3.7 KM	4	6 Number Theory	6	8.4 雙連通分量 & 割點	12	13.1 formula	14
		6.1 basic	6	9 Tree Problem	12	13.1.1 Pick 公式	14
		6.2 bit set	7	9.1 HeavyLight	12	13.1.2 圖論	14
		6.3 cantor expansion	7	9.2 LCA	12	13.1.3 dinic 特殊圖複雜度	14
		6.4 FFT	7	9.3 link cut tree	12	13.1.4 0-1 分數規劃	15
		6.5 find real root	7	9.4 POJ tree	13	13.1.5 學長公式	15
		6.6 FWT	8	10 default	13	13.1.6 基本數論	15
		6.7 LinearCongruence	8	10.1 debug	13	13.1.7 排組公式	15
		6.8 Lucas	8	10.2 ext	13	13.1.8 冪次, 冪次和	15
		6.9 Matrix	8	10.3 IncStack	13	13.1.9 Burnside's lemma	15
		6.10 MillerRobin	8	10.4 input	13	13.1.10 Count on a tree	15
		6.11 NTT	9			14 Интернационал	15
		6.12 Simpson	9			14.1 ganadoQuote	15
		6.13 外星模運算	9			14.2 Интернационал	16
		6.14 數位統計	9			14.3 保佑	16

ACM ICPC Judge Test - CCU- oshiete536senpai

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6 const size_t KB = 1024;
7 const size_t MB = KB * 1024;
8 const size_t GB = MB * 1024;

```

```

9 size_t block_size, bound;
10 void stack_size_dfs(size_t depth = 1) {
11     if (depth >= bound)
12         return;
13     int8_t ptr[block_size]; // 若無法編譯將
14                             // block_size 改成常數
15     memset(ptr, 'a', block_size);
16     cout << depth << endl;
17     stack_size_dfs(depth + 1);
18 }
19
20 void stack_size_and_runtime_error(size_t
21     block_size, size_t bound = 1024) {
22     system_test::block_size = block_size;
23     system_test::bound = bound;
24     stack_size_dfs();
25 }
26
27 double speed(int iter_num) {
28     const int block_size = 1024;
29     volatile int A[block_size];
30     auto begin = chrono::high_resolution_clock
31         ::now();
32     while (iter_num--)
33         for (int j = 0; j < block_size; ++j)
34             A[j] += j;
35     auto end = chrono::high_resolution_clock::
36         now();

```

```

34     chrono::duration<double> diff = end -
35         begin;
36     return diff.count();
37 }
38
39 void runtime_error_1() {
40     // Segmentation fault
41     int *ptr = nullptr;
42     *(ptr + 7122) = 7122;
43 }
44
45 void runtime_error_2() {
46     // Segmentation fault
47     int *ptr = (int *)memset;
48     *ptr = 7122;
49 }
50
51 void runtime_error_3() {
52     // munmap_chunk(): invalid pointer
53     int *ptr = (int *)memset;
54     delete ptr;
55 }
56
57 void runtime_error_4() {
58     // free(): invalid pointer
59     int *ptr = new int[7122];
60     ptr += 1;
61     delete[] ptr;

```

```

62
63 void runtime_error_5() {
64     // maybe illegal instruction
65     int a = 7122, b = 0;
66     cout << (a / b) << endl;
67 }
68
69 void runtime_error_6() {
70     // floating point exception
71     volatile int a = 7122, b = 0;
72     cout << (a / b) << endl;
73 }
74
75 void runtime_error_7() {
76     // call to abort.
77     assert(false);
78 }
79
80 } // namespace system_test
81
82 #include <sys/resource.h>
83 void print_stack_limit() { // only work in
84     Linux
85     struct rlimit l;
86     getrlimit(RLIMIT_STACK, &l);
87     cout << "stack_size = " << l.rlim_cur << "
88         byte" << endl;

```